Smartly Hire

Final Year Project

Session 2022-2026

A project submitted in partial fulfillment of the degree of

BS in Software Engineering

Department of Software Engineering

Faculty of Computer Science & Information Technology

The Superior University, Lahore

Fall 2025

Type (Nature of project)	[✓] **D**evelopment [ ] **R**esearch [ ] **R**&**D**
Area of specialization	
FYP ID	BSSE-FYP-F25-059

Project Group Members

Sr.#	**Reg. #**	**Student Name**	**Email ID**	***Signature**
(i)	SU92-BSSEM-F22-202	Hifza Khalid	su92-bssem-f22-202@superior.edu.pk	
(ii)	SU92-BSSEM-F22-201	Baqir Sultan	su92-bssem-f22-201@superior.edu.pk	
(iii)	SU92-BSSEM-F22-196	Muhammad Zubair	su92-bssem-f22-196@superior.edu.pk	

*The candidates confirm that the work submitted is their own and appropriate credit has been given where reference has been made to work of others

Plagiarism Free Certificate

This is to certify that, I **Hifza Khalid** S/D of **Khalid Nawaz**, group leader of FYP under registration no **BSSE-FYP-F25-059** at Software Engineering Department, The Superior University, Lahore. I declare that my FYP report is checked by my supervisor.

Date: **Nov 24, 2025.** Name of Group Leader: Hifza Khalid Signature:_____

Name of Supervisor: Mr. Sohail Irshad

Designation: Lecturer

Signature: _____

HoD: Dr. Muhammad Aqeel

Signature: _____

Smartly Hire

Change Record

Author(s)	Version	Date	Notes	Supervisor's Signature
Hifza Khalid Baqir Sultan Zubair Iftikhar	1.0	12 Sep, 2025	Initial draft including problem statement, project motivation, and literature survey	
Hifza Khalid Baqir Sultan Zubair Iftikhar	1.1	26 Sep , 2025	Revised requirements and introduction based on supervisor feedback	
Hifza Khalid Baqir Sultan Zubair Iftikhar	1.2	31 Sep, 2025	Updated document after faculty evaluation feedback; improved clarity and scope	
Hifza Khalid Baqir Sultan Zubair Iftikhar	1.3	3 Oct, 2025	Added detailed project plan, WBS, Gantt chart, and responsibility matrix	
Hifza Khalid Baqir Sultan Muhammad Zubair	1.4	10 Oct, 2025	Included system use case model, process workflows, and interview flow definition	
Hifza Khalid Baqir Sultan Muhammad Zubair	1.5	17 Oct, 2025	Added system design models (UML diagrams, database ERD, component layout)	
Hifza Khalid Baqir Sultan Muhammad Zubair	1.6	11 Dec, 2025	Document updated to include implementation details and deployed environment	
Hifza Khalid Baqir Sultan Muhammad Zubair	1.7	16 Feb, 2025	Final optimized report version including evaluation, results, and conclusion	

APPROVAL

PROJECT SUPERVISOR

Comments: _____

_____

Name:_____

Date:_____ Signature:_____

PROJECT MANAGER

Comments: _____

_____

Date:_____ Signature:_____

HEAD OF THE DEPARTMENT

Comments: _____

_____

Date:_____ Signature:_____

Dedication

I dedicate this work to all academic community, industry professionals, and mentors visionaries and innovators who believe in the power of technology to transform how we communicate and connect.

Acknowledgements

*We are thankful to our supervisor **Mr. Sohail Irshad** who has guided us throughout this journey with immense patience and dedication. His expertise, constructive criticism, and continuous support have been instrumental in shaping not just this project, but our approach to innovation and problem-solving. The countless hours spent in discussion, brainstorming, and refinement have proven invaluable to the development of Smartly Hire.*

Executive Summary

Smartly Hire is an AI-powered online interview platform designed to make recruitment secure, efficient, and automated. It provides real-time monitoring of candidates using face detection, emotion recognition, head-pose tracking, and behavioral analysis to ensure fair and proctored interviews. Candidates can apply, attend interviews, and receive instant reports, while recruiters manage jobs, monitor live sessions, and generate detailed performance insights.

Built with a microservices architecture with React and Tailwind for frontend, Node.js and Express for backend, MongoDB for database, and Python-based AI services, Smartly Hire ensures scalability, security, and reliability. The system reduces reliance on human proctors, prevents cheating, and delivers consistent, actionable insights, making remote hiring smarter, faster, and more transparent.

Table of Content

Dedication v
Acknowledgements vi
Executive Summary vii
Table of Content viii
List of Tables xii
Chapter 1 1
Introduction 1
1.1. Background 2
1.2. Motivations and Challenges 2
1.3. Goals and Objectives 3
1.4. Literature Review/Existing Solutions 4
1.5. Gap Analysis 4
1.6. Proposed Solution 5
1.7. Project Plan 6
1.7.1. Work Breakdown Structure 6
1.7.2. Roles & Responsibility Matrix 12
1.7.3. Gantt Chart 12
1.8. Report Outline 13
1.9. Empathy Map 15
Chapter 2 16
Software Requirement Specifications 16
2.1. Introduction 17
2.1.1. Purpose 17
2.1.2. Document Conventions 17
2.1.3. Intended Audience and Reading Suggestions 18
2.1.4. Product Scope 18
2.2. Overall Description 18
2.2.1. Product Perspective 18
2.2.2. Product Functions: 19
2.2.3. User Classes and Characteristics 19
2.2.4. Operating Environment 20
2.2.5. Design and Implementation Constraints 20
2.2.6. User Documentation 20
2.2.7. Assumptions and Dependencies 20
2.3. External Interface Requirements 21
2.3.1. User Interfaces 21
2.3.2. Hardware Interfaces 21
2.3.3. Software Interfaces 22
2.3.4. Communications Interfaces 22
2.4. System Features 22
2.4.1. User Authentication & Management 22
2.4.1.1. Description and Priority 22

2.4.1.2. Stimulus/Response Sequences 22
2.4.1.3. Functional Requirements Table 23
2.4.2. Admin Functional Features 23
2.4.2.1. Description and Priority 23
2.4.2.2. Stimulus/Response Sequences 23
2.4.2.3. Functional Requirements 24
2.4.3. Candidate Functional Features 24
2.4.4. Browser Permission & Video Access 25
2.4.5. Interview Question Handling 25
2.4.6 AI Proctoring & Monitoring 26
2.4.7 Auto-Submission & Notifications 27
2.4.8 Report Generation 27
2.4.9 System & Data Management 28
2.4.10 External Services 28
2.5. Nonfunctional Requirements 29
2.5.1. Performance Requirements 29
2.5.2. Safety Requirements 30
2.5.3. Security Requirements 30
2.5.4. Usability Requirements 30
2.5.5. Reliability Requirements 31
2.5.6. Maintainability/Supportability Requirements 31
2.5.7. Portability Requirements 32
2.5.8. Efficiency Requirements 32
2.6. Domain Requirements 33
Chapter 3 34
Use Case Analysis 34
3.1. Use Case Model 35
3.2. Use Cases Description 37
Chapter 4 45
System Design 45
4.1. Architecture Diagram 46
4.2. Domain Model 47
4.3. Entity Relationship Diagram with data dictionary 49
4.4. Class Diagram 59
4.5. Sequence Diagram 61
4.6. Operation contracts 64
4.7. Activity Diagram 69
4.8. State Transition Diagram 71
4.9. Component Diagram 73
4.10. Deployment Diagram 75
4.11. Data Flow diagram 76
Chapter 5 78
Implementation 78
5.1. Important Flow Control/Pseudo codes 79

5.2. Components, Libraries, Web Services and stubs 81
5.3. Deployment Environment 82
5.4. Tools and Techniques 83
5.5. Best Practices / Coding Standards 83
5.6. Version Control 84
Chapter 6 86
Testing and Evaluation 86
6.1 Use Case Testing 87
6.2 Equivalence partitioning 88
6.3 Boundary value analysis 88
6.4 Data flow testing 89
6.5 Unit testing 89
6.6 Integration testing 90
6.7 Performance testing 90
6.8 Stress Testing 91
Chapter 7 92
Summary, Conclusion and Future Enhancements 92
7.1. Project Summary 93
7.2. Achievements and Improvements 93
7.3. Critical Review 93
7.4. Lessons Learnt 94
7.5. Future Enhancements/Recommendations 94
Appendices 95
Appendix A: Information / Promotional Material 96
A.1. Broacher 96
A.2. Flyer 98
A.3. Standee 98
A.4. Branding Guidelines 99
Appendix B: Sample Screenshots 100
B.1. Admin Dashboard Screenshots 101
B.2. Candidate Interface Screenshots 103
Reference and Bibliography 105
Index 108
Index 109

List of Figures

1.1 Gap Analysis 4
1.2 Work Breakdown Structure 6
1.3 Roles & Responsibility Matrix 12
2.1 User Classes and Characteristics 19
2.2 User Interfaces 21
2.3 Hardware Interfaces 21
2.4 Functional Requirements 23
2.5 Admin Functional Features 24
2.6 Candidate Functional Features 24
2.7 Browser Permission & Video Access (Functional Requirements) 25
2.8 Interview Question Handling (Functional Requirements) 25
2.9 AI Proctoring & Monitoring (Functional Requirements) 26
2.10 Auto-Submission & Notifications (Functional Requirements) 27
2.11 Report Generation (Functional Requirements) 28
2.12 System & Data Management (Functional Requirements) 28
2.13 External Services (Functional Requirements) 29
4.1 Entity: users 51
4.2 Entity: Jobs 52
4.3 Entity: Applications 53
4.4 Entity: Interview Sessions 54
4.5 Entity: Questions 55
4.6 Entity: Answers 56
4.7 Entity: Violations 57
4.8 Entity: Reports 58
4.9 Relationships (Data Dictionary Linkage) 59
4.10 User Operations 64
4.11 Job Operations (User Authentication) 65
4.12 Job Operations (Admin) 65
4.13 Application Operations 66
4.14 InterviewSession Operations 67
4.15 Question & Answer Operations 67
4.16 Violation Operations 68
4.17 Report Operations 69

List of Tables

1.1 Gap Analysis 4
1.2 Work Breakdown Structure 6
1.3 Roles & Responsibility Matrix 12
2.1 User Classes and Characteristics 19
2.2 User Interfaces 21
2.3 Hardware Interfaces 21
2.4 Functional Requirements 23
2.5 Admin Functional Features 24
2.6 Candidate Functional Features 24
2.7 Browser Permission & Video Access (Functional Requirements) 25
2.8 Interview Question Handling (Functional Requirements) 25
2.9 AI Proctoring & Monitoring (Functional Requirements) 26
2.10 Auto-Submission & Notifications (Functional Requirements) 27
2.11 Report Generation (Functional Requirements) 28
2.12 System & Data Management (Functional Requirements) 28
2.13 External Services (Functional Requirements) 29
4.1 Entity: users 51
4.2 Entity: Jobs 52
4.3 Entity: Applications 53
4.4 Entity: Interview Sessions 54
4.5 Entity: Questions 55
4.6 Entity: Answers 56
4.7 Entity: Violations 57
4.8 Entity: Reports 58
4.9 Relationships (Data Dictionary Linkage) 59
4.10 User Operations 64
4.11 Job Operations (User Authentication) 65
4.12 Job Operations (Admin) 65
4.13 Application Operations 66
4.14 InterviewSession Operations 67
4.15 Question & Answer Operations 67
4.16 Violation Operations 68
4.17 Report Operations 69

Chapter 1

Introduction

Chapter 1: Introduction

This chapter provides an overview of **Smartly Hire**, a web-based AI-assisted recruitment platform, outlining the background of the problem, motivation for developing an automated recruitment solution, and the modern challenges associated with remote hiring environments. It reviews existing tools, identifies gaps in traditional recruitment workflows, and presents the proposed solution. The chapter concludes with the project plan, including the WBS structure, roles and responsibility matrix, Gantt chart outline, and empathy map, which collectively define the operational framework for the system's development.

1.1. Background

With the rapid digital transformation of organizational processes, recruitment has shifted from traditional in-person interviews to remote, technology-driven hiring models [7]. While online interviews reduce administrative overhead, expand candidate reach, and accelerate hiring cycles, they introduce challenges such as:

- Impersonation or proxy interviews
- Use of unauthorized devices (mobile phones, books, or notes)
- External assistance
- Lack of reliable candidate behavior analysis
- Manual, biased, or inconsistent evaluation

To address these challenges, organizations are adopting AI-driven solutions for live proctoring and automated assessment [7]. Smartly Hire is designed to provide a secure, AI-enabled recruitment platform capable of real-time monitoring, automated analysis, and fair evaluation, thereby improving integrity and reliability in remote hiring.

1.2. Motivations and Challenges

Motivations:

- Growing demand for scalable remote hiring tools
- Need for automated evaluation to reduce human bias [7]

- Requirement for secure online interview environments
- Rising impersonation and cheating attempts during remote interviews
- Recruitment teams require faster, data-driven decision-making

Challenges:

Developing such a system involves:

- Real-time AI detection of candidate presence, attention, and prohibited objects using computer vision models [2], [3], [4]
- Processing large volumes of live video and behavioral data with minimal latency using optimized ML frameworks [5], [6]
- Designing a clean, intuitive interface for both candidates and recruiters
- Ensuring cybersecurity, data privacy, and seamless cloud deployment
- Maintaining accuracy and generalization across different devices and environments [7]

1.3. Goals and Objectives

Main Goal:

To develop **Smartly Hire**, an AI-assisted recruitment platform that automates hiring, enhances security, and ensures fair evaluation in remote interview environments.

Specific Objectives:

- Build a fully responsive frontend for both Admin (Recruiter/HR) and Candidate
- Implement secure authentication with role-based access
- Provide tools for job posting, application management, and interview scheduling
- Integrate ML services for live proctoring:
    - Face detection and tracking using MediaPipe [3]
    - Multiple face detection using deep learning models [4]
    - Head pose analysis using computer vision techniques [17]
    - Suspicious object detection (cell phone, book, paper) using object detection models [4], [18]
- Automatically detect:

  - Candidate leaving camera view
  - Multiple faces
  - Unauthorized devices
  - Looking away or distraction events
- Generate structured interview reports with integrity scores, analytics, and optional AI evaluation
- Ensure system scalability through modern cloud deployment

1.4. Literature Review/Existing Solutions

Current platforms such as HireVue, Mettl, TestGorilla, and HackerRank offer automated scoring, basic proctoring, or skill testing [7]. However:

- Real-time cheating detection is limited
- Most systems focus on a single capability rather than full interviews
- Emotion and behavioral analytics are often restricted or expensive [7]
- Systems lack transparency, customization, and multi-modal monitoring
- Costs remain prohibitive for universities and startups

Research in facial behavior analysis and deep learning-based monitoring systems shows that combining multiple modalities (face, emotion, and object detection) significantly improves reliability and accuracy in automated evaluation systems [7], [8].

1.5. Gap Analysis

1.1 Gap Analysis

Area	Existing Systems	Identified Gap
Proctoring Depth	Limited monitoring	No unified real-time multi-modal AI

AI Models	Either object detection OR emotion	Lack of combined face, emotion, head-pose, device detection
Reporting	Basic Scoring	Need for analytics, charts, PDF reports
Customization	Restricted thresholds	No flexible rules/weights per organization
Cost	High subscription plans	Not feasible for small institutions
Scalability	Manual supervision still required	Need for fully automated pipeline

These gaps reinforce the need for a unified, AI-assisted online recruitment platform.

1.6. Proposed Solution

Smartly Hire is a cloud-based, AI-assisted recruitment platform designed to ensure fairness, security, and transparency in online assessments. The system integrates secure authentication, job management, automated interview workflows, and real-time AI-assisted proctoring using lightweight ML models.

The platform captures candidate webcam frames and behavioral cues for integrity evaluation. Instead of heavy models, it leverages MediaPipe + COCO-SSD + TensorFlow.js for efficient live monitoring, enabling real-time analysis even on standard devices [2], [3], [5].

AI/ML Components Used:

- Face Detection & Tracking: MediaPipe FaceMesh (468 points) [2], [3]
- Object Detection: COCO-SSD / YOLO-based models [4], [18]
- Emotion Detection: Mini-Xception CNN [1]
- Head Pose Estimation: SolvePnP-based approach [17]
- Integrity Score Computation: Weighted deduction system

1.7. Project Plan

The project follows a structured development approach: requirements gathering, system design, frontend/backend development, ML microservices integration, testing, and deployment. Each phase has defined tasks, timelines, and responsibilities.

1.7.1. Work Breakdown Structure

2.2 Work Breakdown Structure

WBS #	WBS Item	Deliverable	Activity #	Activity to Complete the Deliverable	Duration (Days)	Responsible Team Member(s) & Role(s)
1	**Requirements & Analysis**	Requirements Specification	1.1	Collect functional requirements (admin, candidate, proctoring)	7	Baqir (Lead), Hifza & Zubair (Support)
			1.2	Collect non-functional requirements (security, performance, usability)	5	Baqir (Lead)
			1.3	Prepare Software Requirements	10	Baqir (Lead), Hifza &

				Specification (SRS)		Zubair (Support)
			1.4	Prepare use cases & initial UML diagrams	6	Baqir (Lead), Hifza & Zubair (Support)
2	**System Design**	Architecture & Database	2.1	Define overall architecture (frontend, backend, ML microservices)	7	Baqir (Lead)
			2.2	Design database schema (ERD, collections, relations)	6	Zubair (Lead), Hifza & Baqir (Support)
			2.3	Design module-level components (auth, jobs, interviews, reports)	8	Baqir (Lead)
			2.4	Prepare all design diagrams (Architecture, Domain Model, ERD, Class,	12	Shared (All)

				Sequence, Activity)		
3	**Frontend Development**	Fully Functional UI	3.1	Setup Smartly Hire React Project	4	Hifza (Lead), Baqir & Zubair (Support)
			3.2	Implement authentication pages (login/signup for Admin & Candidate)	6	Hifza (Lead)
			3.3	Admin dashboard UI	8	Hifza (Lead)
			3.4	Job management screens	7	Hifza (Lead)
			3.5	Candidate job listings & applications	8	Hifza (Lead)
			3.6	Proctored interview screen (questions, timer, editor, webcam capture)	14	Hifza (Lead)

			3.7	Integrate charts (Chart.js) for reports	5	Hifza (Lead)
4	**Backend Development (Node.js + MongoDB**)	Fully Functional APIs	4.1	Setup Express server & MongoDB connection	4	Hifza (Lead)
			4.2	Implement authentication module (JWT + roles)	6	Hifza (Lead)
			4.3	Job management APIs	6	Hifza (Lead)
			4.4	Applications & interview scheduling APIs	6	Hifza (Lead)
			4.5	Interview session & answer storage APIs	7	Hifza (Lead)
			4.6	Proctoring event logging APIs	6	Hifza (Lead)
			4.7	Report generation APIs (JSON + PDF export)	8	Hifza (Lead)

5	**ML & Proctoring Services**	Teal-time Proctoring & Evaluation	5.1	Integrate MediaPipe Face Detection in frontend	6	Hifza (Lead), Baqir (Support)
			5.2	Integrate MediaPipe FaceMesh for 468-point tracking	10	Hifza (Lead), Baqir & Zubair (Support)
			5.3	Integrate TensorFlow.js COCO-SSD for object detection	6	Hifza (Lead)
			5.4	Implement Head Pose Estimation (SolvePnP using mesh points)	12	Hifza (Lead)
			5.5	Implement MAR-based mouth movement detection	6	Hifza (Lead)
			5.6	Violation Engine – combine ML outputs into integrity scoring	7	Baqir (Lead), Hifza (Support)
6	**Integration, Testing & Deployment**	Fully Working System	6.1	Integrate frontend with baxckend APIs	7	Shared (All)

			6.2	Functional testing of all flows	10	Shared (All)
			6.3	Performance & reliability testing	7	Shared (All)
			6.4	Deploy frontend (Vercel/Netlify)	3	Baqir (Lead), Hifza & Zubair (Support)
			6.5	Deploy backend (Render/Railway)	3	Baqir (Lead), Hifza & Zubair (Support)
			6.6	Deploy ML services (cloud VM / containers)	5	Baqir (Lead), Hifza & Zubair (Support)
7	Documentation & Presentation	Final Project Deliverables	7.1	Final SRS & design chapters	7	Shared (All)
			7.2	User manuals (admin + candidate)	6	Shared (All)
			7.3	Final report with results, charts, screenshots	10	Shared (All)

			7.4	Presentation slides & demo script	5	Zubair (Lead), Hifza & Baqir (Support)

1.7.2. Roles & Responsibility Matrix

3.3 Roles & Responsibility Matrix

Role / Task	**Hifza Khalid** (Team Lead)	**Baqir Sultan**	**Zubair Iftikhar**
Requirements & SRS	Support	Lead	Support
Frontend Development	Lead	Support	Support
Backend Development	Lead	Support	Support
ML Microservices Face Detection, FaceMesh, Object Detection, Head Pose, EAR, MAR)	Lead	Support	Support
Database Design & Integration	Support	Support	Lead
Testing & Integration	Shared	Shared	Shared
Deployment (Cloud / Hosting)	Support	Lead	Support
Documentation	Shared	Shared	Shared
Presentation & Reporting	Support	Support	Lead

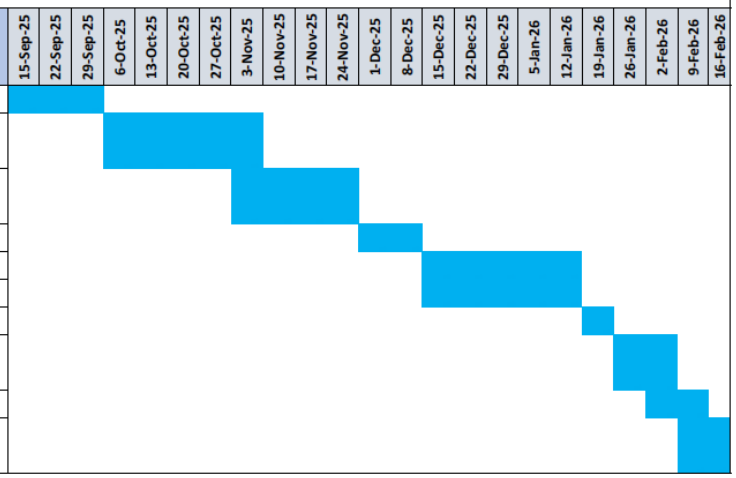
1.7.3. Gantt Chart

The 24-week project is organized into ten phases. It starts with requirements gathering and SRS preparation, then moves into system design (overall architecture, database, and UI components). Next, we implement the core frontend screens, design the proctored interview UI and charts, and set up the backend with all core APIs. In the following phases, we integrate the proctoring-related AI modules, connect the frontend with the backend, and thoroughly

test the system for both functionality and performance. Finally, we deploy the frontend, backend, and ML services, and complete the documentation and final reporting.

Gantt Chart (Smartly Hire)

Task ID	Task Name	Start Date	End Date
1	Requirements & SRS	15-Sep-25	5-Oct-25
2	System Design (Architecture, DB, UI Components)	6-Oct-25	4-Nov-25
3	Frontend Core Screens (React Web App)	3-Nov-25	29-Nov-25
4	Proctored Interview UI & Charts	1-Dec-25	11-Dec-25
5	Backend Setup & Core APIs	15-Dec-25	17-Jan-26
6	Proctoring AI Modules Integration	15-Dec-25	16-Jan-26
7	Frontend–Backend Integration	19-Jan-26	23-Jan-26
8	System Testing (Functional & Performance)	26-Jan-26	6-Feb-26
9	Setup React project	2-Feb-26	11-Feb-26
10	Deployment (Frontend, Backend, ML Services)	9-Feb-26	20-Feb-26



1.1 Gantt Chart

1.8. Report Outline

The final year project report for Smartly Hire is structured into seven major chapters that collectively describe the complete system, its problem domain, design, implementation, evaluation, outcomes, and future direction.

•**Chapter 1 (Introduction):** Presents recruitment challenges in remote hiring, motivation for secure and scalable automated interviews, gap analysis in current hiring tools, and overview of the Smartly Hire platform with its two core actors: Admin and Candidate. It also outlines the project planning artifacts including WBS, responsibility mapping, Gantt chart, and report structure.

• **Chapter 2 (Software Requirement Specifications):** Defines detailed functional, non-functional, security and domain-specific requirements of Smartly Hire. It describes system constraints, interfaces, performance benchmarks, user behavior assumptions, and AI-enabled services used for interview monitoring and evaluation.

• **Chapter 3 (Use Case Analysis):** Models interaction scenarios between hiring Admins and Candidates in Smartly Hire using structured use cases. It covers user authentication, job selection,

interview flow, proctoring triggers, data storage, and report generation, ensuring traceability across system requirements.

• **Chapter 4 (System Design):** Describes the architectural blueprints of Smartly Hire, including UML and system modeling diagrams. This involves high-level architecture components, domain structure, database entities, behavioral interaction sequences, component integration layouts, cloud execution environment, module contracts, and system states.

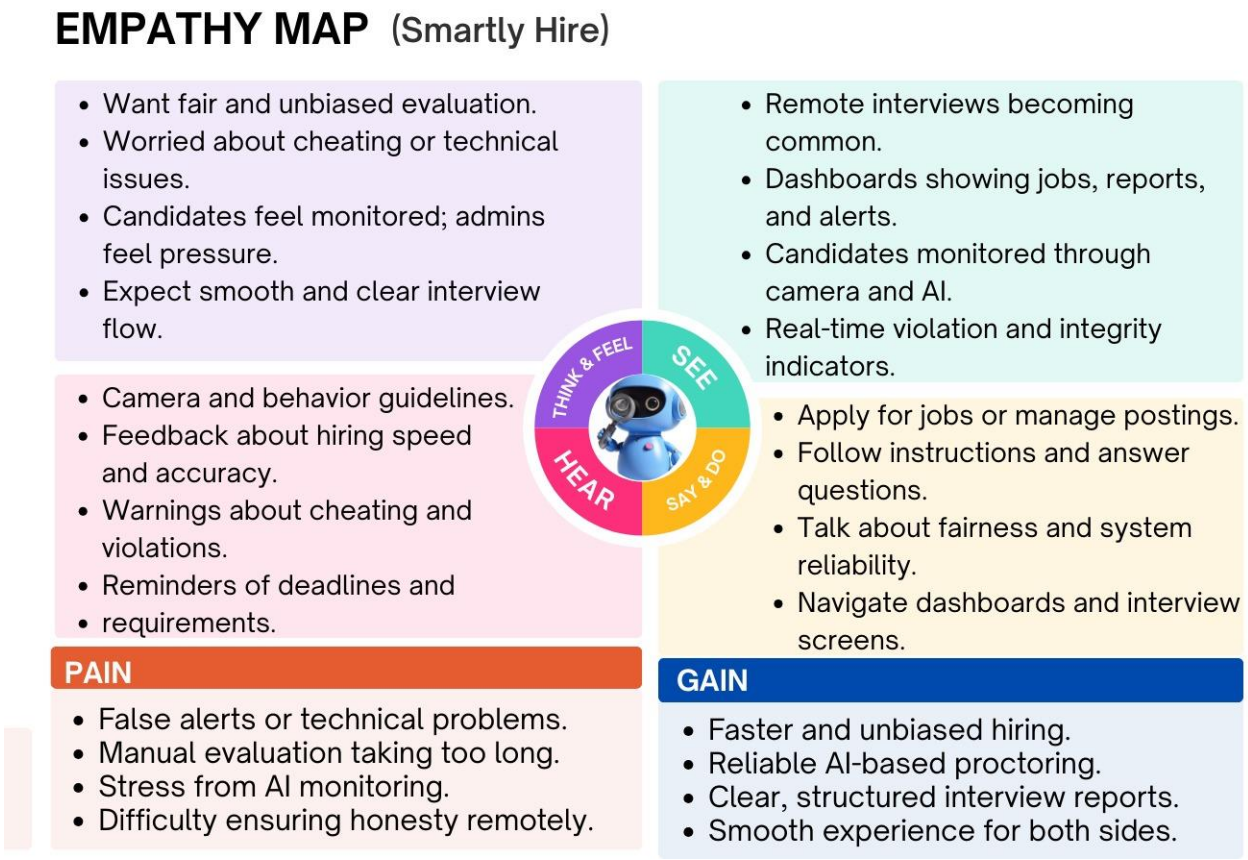
• **Chapter 5 (Implementation):** Demonstrates actual development of Smartly Hire using its complete tech stack. It covers frontend and backend API integration, AI proctoring module execution inside the browser, database persistence, development tools, coding standards, deployment environment, and version control practices.

• **Chapter 6 (Testing and Evaluation):** Presents the complete testing process including use case testing, equivalence partitioning, boundary value analysis, data flow testing, unit testing, integration testing, performance testing, and stress testing. It confirms that authentication, job creation, AI question generation, proctoring, and result evaluation work accurately under normal and heavy load conditions.

• **Chapter 7 (Summary, Conclusion & Future Enhancements):** Provides a project summary, key achievements, critical review of system limitations, lessons learned during development, and future recommendations including audio voice-to-text, color-blind accessible themes, live streaming, and mobile application support.

Primary report modules communicate securely through REST and internal microservice stubs to enable real-time candidate monitoring and AI-based evaluation. This report format ensures traceable requirements, modular design clarity, expert-level implementation, thorough validation, and recruiter-ready documented outputs with integrity-focused evaluation support.

1.9. Empathy Map



1.2 Empathy Map

Chapter 2

Software Requirement Specifications

Chapter 2: Software Requirement Specifications

2.1. Introduction

This chapter defines the Software Requirements Specification (SRS) for the Smartly Hire system. It describes the system's functional and non-functional requirements, external interfaces, and operational constraints. The SRS provides a detailed understanding of system behavior, ensuring that developers, stakeholders, and evaluators have a clear and structured specification of the platform.

2.1.1. Purpose

The purpose of Smartly Hire is to develop an AI-assisted recruitment platform that automates hiring processes, enhances security during online interviews, and provides fair and data-driven evaluation.

The system integrates:

- Real-time facial and behavior analysis using MediaPipe Face Detection and FaceMesh [2], [3]
- Emotion detection using Mini-Xception CNN [1], [7]
- Object detection for cheating detection using deep learning models [4], [18]
- Automated proctoring violation detection based on behavioral analytics [7]

2.1.2. Document Conventions

Fonts & Styles: Headings use bold and numbered styles; requirement statements use numbered tags (e.g., REQ-SF1-1).

Requirement Priorities:

- **Must:** Essential for correct system operation
- **Should:** Highly desirable, not critical
- **May:** Optional or enhancement features

Traceability: Every requirement is uniquely tagged for traceability.

T**erminology:**

- "Candidate" refers to the person taking the interview.
- "Admin/Recruiter" refers to the hiring manager or HR user.

2.1.3. Intended Audience and Reading Suggestions

- **Developers:** For implementing frontend, backend, and AI services.
- **Project Managers:** To track requirement coverage and timelines.
- **Testers:** To validate functionality, UI, AI proctoring accuracy, and reporting.
- **Documentation Writers:** To prepare user manuals and guides.
- **End Users:** Admins and candidates to understand system functionalities.

Reading Sequence:

1. Overview & system context (Sections 2.2 & 2.3)
2. Functional requirements (Section 2.4)
3. Non-functional and domain requirements (Sections 2.5 & 2.6)

2.1.4. Product Scope

Smartly Hire is a cloud-based platform enabling secure, monitored online interviews.

Key objectives:

- Real-time AI monitoring of candidate behavior (face, emotions, attention, multiple screens).
- Automated recording and reporting of candidate performance and violations.
- Streamlined job and interview management for recruiters.
- Supports multiple question formats: coding, and short questions.

Benefits:

- Reduces human proctor dependency.
- Ensures fairness and compliance in remote interviews.
- Provides actionable insights to recruiters via reports and dashboards.

2.2. Overall Description

2.2.1. Product Perspective

Smartly Hire is a **self-contained platform for secure remote interviews**, built using a **microservices architecture**:

- **Frontend:** React Vite + Tailwind (UI/UX, dashboards, forms)

- **Backend:** Node.js + Express (API layer, authentication, job/interview management)
- **Database:** MongoDB Atlas (users, jobs, interviews, violations, reports)
- **AI Services:** Python + FastAPI microservices (Face Detection, FaceMesh, Emotion, Head Pose, EAR/MAR)

2.2.2. Product Functions:

Key functions include:

- Candidate registration, job application, and interview scheduling.
- Live AI proctoring (emotion, head-pose, cheating detection).
- Real-time alerts for violations.
- Performance analytics and PDF reports.
- Admin dashboards with charts and insights.

2.2.3. User Classes and Characteristics

2.1 User Classes and Characteristics

User Class	**Description**	**Frequency**	**Tech Skill**
Admin/Recruiter	Manages jobs, reviews candidates, monitors interview reports	Frequent	Moderate
Candidate	Attends interviews, answers questions, consents to AI proctoring	Occasional	Basic
System/AI	Automated proctoring agent, evaluates candidate behavior	Continuous	N/A

2.2.4. Operating Environment

The system operates in a web-based environment and supports:

- Modern browsers (Chrome, Edge, Firefox)
- Internet-enabled devices (desktop/laptop)
- Cloud-based deployment

AI components require:

- Webcam access
- Stable internet connection
- Real-time processing capability using optimized frameworks [5], [6]

2.2.5. Design and Implementation Constraints

- System must ensure low latency for real-time proctoring [5]
- AI models must be lightweight and efficient for browser-based execution [6]
- Must comply with data privacy and security standards
- Dependency on third-party libraries such as MediaPipe and TensorFlow [3], [5]

2.2.6. User Documentation

Documentation provides guidance for all system users:

- **Candidates:** Registration, login, dashboard, job listings, interview participation, timers, proctoring alerts, device setup, viewing reports.
- **Admins:** Dashboard navigation, job management, AI-based interview report interpretation, violation monitoring, audit trails.
- **Developers:** Frontend, backend, database, ML microservices integration, REST API references, deployment instructions, monitoring/maintenance.

2.2.7. Assumptions and Dependencies

- Users have access to a working webcam and internet connection
- Third-party libraries remain stable and supported [3], [5]

- Cloud infrastructure is available for deployment

2.3. External Interface Requirements

2.3.1. User Interfaces

2.2 User Interfaces

User Type	UI Component	Description / Features
Candidate	Interview Screen	Displays question area, timer, answer field (MCQ, coding, short), webcam preview, and live proctoring alerts.
Admin/Recruiter	Dashboard	Overview cards, charts, violation reports, candidate details, job management.
All Users	Standard Components	Navbar, breadcrumbs, standard buttons, error messages, validation feedback.

- Responsive design optimized for desktop, laptop, and tablet.
- Consistent layout using React + Tailwind.

2.3.2. Hardware Interfaces

2.3 Hardware Interfaces

User Type	**Hardware Requirement**	**Description**
Candidate	Laptop/Desktop	Webcam (≥720p), multi-screen triggers violation.
Admin	Laptop/Desktop	Modern browser (Chrome, Firefox, Edge) with internet

		access. No special hardware required.

2.3.3. Software Interfaces

- **Frontend:** React + Vite + Tailwind, WebRTC, Web Speech API.
- **Backend:** Node.js + Express REST APIs (auth, job/interview management, logging).
- **Database:** MongoDB Atlas (users, jobs, sessions, violations, reports).
- **ML Microservices:** Python + FastAPI (face detection, head-pose, emotion, EAR/MAR, violation detection).

2.3.4. Communications Interfaces

- **Frontend ↔ Backend:** HTTPS REST APIs (JSON data).
- **Real-time notifications:** WebSockets or long-polling.
- **Backend ↔ AI Microservices:** Secure internal HTTP/HTTPS calls.
- **Port assignments**: Frontend/Backend: 443 (prod), 5173/5000 (dev); AI services: 8001–8003.

2.4. System Features

2.4.1. User Authentication & Management

2.4.1.1. Description and Priority

Manages user registration, login, session, and password security. Priority: High (core functionality).

2.4.1.2. Stimulus/Response Sequences

- Candidate/Admin opens registration page → enters details → account created.
- Candidate/Admin logs in → system authenticates → grants access to dashboard.
- Inactive users → session auto-logout.

2.4.1.3. Functional Requirements Table

2.4 Functional Requirements

Req ID	Req Definition	Want	Stakeholder
REQ-AUTH-1	System shall allow Candidate/Admin to create an account with name, email, password	Must	Candidate, Admin
REQ-AUTH-2	System shall authenticate users via email/password	Must	Candidate, Admin
REQ-AUTH-3	System shall enforce strong password rules and hash passwords	Must	Admin, Security
REQ-AUTH-4	System shall ensure unique email addresses	Must	Admin, Candidate
REQ-AUTH-5	System shall maintain active sessions and auto-logout inactive users	Must	Candidate, Admin

2.4.2. Admin Functional Features

2.4.2.1. Description and Priority

Allows Admins to manage jobs, candidates, interviews, and view reports. Priority: High

2.4.2.2. Stimulus/Response Sequences

- Admin logs in → navigates to dashboard → views summary.
- Admin selects job → create/edit/delete → job data updated.

- Admin views candidate list → reviews interview reports/violations.

2.4.2.3. Functional Requirements

2.5 Admin Functional Features

Req ID	Req Definition	Want	Stakeholder
REQ-ADMIN-1	View admin dashboard summarizing jobs, candidates, interview statuses	Must	Admin
REQ-ADMIN-2	Create job posting with job details	Must	Admin
REQ-ADMIN-3	Edit existing job posting	Must	Admin
REQ-ADMIN-4	Delete a job posting	Must	Admin
REQ-ADMIN-5	Generate AI-based job description	Want	Admin
REQ-ADMIN-6	View candidate list per job	Must	Admin
REQ-ADMIN-7	View AI-generated interview reports	Must	Admin
REQ-ADMIN-8	View violations detected during interviews	Must	Admin

2.4.3. Candidate Functional Features

2.4.3.1 Description and Priority:

Supports candidate dashboard, job application, interview participation. **Priority: High**

2.4.3.2 Stimulus/Response Sequences:

1. Candidate logs in → sees dashboard → selects job → applies.
2. Candidate uploads resume → starts interview → answers questions → submits.

2.4.3.3 Functional Requirements:

2.6 Candidate Functional Features

Req ID	Req Definition	Wish/Want	Stakeholder
REQ-CAND-1	Display available job postings	Must	Candidate
REQ-CAND-2	Apply for a selected job	Must	Candidate
REQ-CAND-3	Upload resume as part of application	Must	Candidate
REQ-CAND-4	Start assigned interview	Must	Candidate

REQ-CAND-5	Answer text/video questions	Must	Candidate
REQ-CAND-6	Manually submit interview	Must	Candidate

2.4.4. Browser Permission & Video Access

2.4.4.1 Description and Priority:

Ensures proper browser permissions for camera. **Priority: High**

2.4.4.2 Stimulus/Response Sequences:

1. Candidate opens interview page → system checks camera → alerts if denied.

2.4.4.3 Functional Requirements:

2.7 Browser Permission & Video Access (Functional Requirements)

Req ID	Req Definition	Wish/Want	Stakeholder
REQ-BROWSER-1	Check camera permission	Must	Candidate
REQ-BROWSER-2	Alert candidate if permission denied	Must	Candidate

2.4.5. Interview Question Handling

2.4.5.1 Description and Priority:

Handles loading, presenting, and recording interview questions. **Priority: High**

2.4.5.2 Stimulus/Response Sequences:

1. Interview starts → questions loaded → system reads aloud if enabled → candidate responds → data stored.

2.4.5.3 Functional Requirements:

2.8 Interview Question Handling (Functional Requirements)

Req ID	Req Definition	Wish/Want	Stakeholder
REQ-Q-1	Load job-specific interview questions before start	Must	Candidate, Admin

REQ-Q-2	Optional text-to-speech reading of questions	Want	Candidate
REQ-Q-3	Record video/text responses	Must	Candidate, Admin

2.4.6 AI Proctoring & Monitoring

2.4.6.1 Description and Priority:

Real-time candidate monitoring for cheating detection and behavior analysis. **Priority: High**

2.4.6.2 Stimulus/Response Sequences:

1. Candidate starts interview → webcam active → system monitors → violations logged → threshold triggers alerts/auto-submit.

2.4.6.3 Functional Requirements:

2.9 AI Proctoring & Monitoring (Functional Requirements)

Req ID	Req Definition	Wish/Want	Stakeholder
REQ-PROCTOR-1	Continuous monitoring via webcam	Must	Admin, Candidate
REQ-PROCTOR-2	Face detection using MediaPipe	Must	System
REQ-PROCTOR-3	Extract 468 facial landmarks	Must	System
REQ-PROCTOR-4	Detect candidate emotions via Mini-Xception	Must	Admin
REQ-PROCTOR-5	Head pose estimation	Must	Admin
REQ-PROCTOR-6	Eye blink detection	Must	Admin
REQ-PROCTOR-7	Mouth movement detection	Must	Admin
REQ-PROCTOR-8	Detect anomalies (phone, extra screen/person, face missing)	Must	Admin
REQ-PROCTOR-9	Log detected violations	Must	Admin
REQ-PROCTOR-10	Threshold-based auto-actions	Must	Admin, Candidate

2.4.7 Auto-Submission & Notifications

2.4.7.1 Description and Priority:

Automatically submits interview if violations occur or inactivity is detected. **Priority: High**

2.4.7.2 Stimulus/Response Sequences:

1. Candidate exceeds violation threshold → interview auto-submitted → candidate notified.

2.4.7.3 Functional Requirements:

2.10 Auto-Submission & Notifications (Functional Requirements)

Req ID	Req Definition	Wish/Want	Stakeholder
REQ-AUTO-1	Auto-submit interview on violation threshold	Must	Candidate, Admin
REQ-AUTO-2	Auto-submit if candidate disappears from webcam	Must	Candidate, Admin
REQ-AUTO-3	Auto-submit on repeated tab/browser switches	Must	Candidate, Admin
REQ-AUTO-4	Notify candidate after auto-submission	Must	Candidate

2.4.8 Report Generation

2.4.8.1 Description and Priority:

Generates AI-based final interview reports for admin/candidate review. **Priority: High**

2.4.8.2 Stimulus/Response Sequences:

1. Interview completed → system computes score → report generated → stored for viewing.

2.4.8.3 Functional Requirements:

2.11 Report Generation (Functional Requirements)

Req ID	Req Definition	Wish/Want	Stakeholder
REQ-REPORT-1	Generate final AI-based report	Must	Admin, Candidate
REQ-REPORT-2	Compute overall score (emotion, head pose, answers, violations)	Must	Admin
REQ-REPORT-3	Store report for review	Must	Admin, Candidate
REQ-REPORT-4	Allow Admin to view all reports	Must	Admin
REQ-REPORT-5	Allow Candidate to view own report	Must	Candidate

2.4.9 System & Data Management

2.4.9.1 Description and Priority:

Ensures secure storage, backup, and encryption of candidate/interview data.

Priority: High

2.4.9.2 Stimulus/Response Sequences:

1. Candidate responses recorded → system saves in DB → backups maintained → data encrypted.

2.12 System & Data Management (Functional Requirements)

Req ID	Req Definition	Wish/Want	Stakeholder
REQ-DATA-1	Save interview video/text responses	Must	Admin, Candidate
REQ-DATA-2	Save job applications and resumes	Must	Admin, Candidate
REQ-DATA-3	Encrypt sensitive data	Must	Security, Admin
REQ-DATA-4	Maintain secure backups of data	Must	Admin

2.4.10 External Services

2.4.10.1 Description and Priority:

Integrate browser APIs and AI/ML services for proctoring and speech functionality.

Priority: High

2.4.10.2 Stimulus/Response Sequences:

1. Candidate starts interview → browser APIs accessed → ML services invoked for monitoring → responses processed.

2.4.10.3 Functional Requirements:

2.13 External Services (Functional Requirements)

Req ID	Req Definition	Wish/Want	Stakeholder
REQ-EXT-1	Integrate browser camera APIs	Must	Candidate, System
REQ-EXT-2	Use Web Speech API for speech synthesis/recognition	Want	Candidate
REQ-EXT-3	Utilize MediaPipe Face Detection	Must	System
REQ-EXT-4	Utilize MediaPipe FaceMesh	Must	System
REQ-EXT-5	Run Mini-Xception CNN for emotion classification	Must	Admin, System

2.5. Nonfunctional Requirements

2.5.1. Performance Requirements

2.5.1.1. **ML Frame Processing:** MediaPipe + Mini-Xception must process webcam frames within 3–5 seconds to ensure accurate cheating detection, emotion recognition, and head-pose analysis.

2.5.1.2. **Frontend Responsiveness:** All frontend actions including login, navigating jobs, starting interviews, and moving to the next question must respond within 2 seconds under normal load.

2.5.1.3. **Concurrent Interview Support:** The system must handle at least 50 simultaneous interviews without any performance degradation.

2.5.1.4. **Report & Analytics Load:** Candidate answers, reports, and analytics should load within 3 seconds.

2.5.1.5. **Caching:** Frequently accessed data such as interview questions and job listings must be cached to reduce database queries and ensure retrieval within 2 seconds.

2.5.1.6. **Scalability:** The system architecture must support horizontal scaling to accommodate increased candidate loads and concurrent interview sessions.

2.5.2. Safety Requirements

2.5.2.1. **Device Safety:** Candidate device such as webcams must operate within standard voltage limits, and the system must not issue any commands that could damage them.

2.5.2.2. **Obstruction Alerts:** The system must notify candidates if their webcam is obstructed to prevent false proctoring violations.

2.5.2.3. **Auto-Save:** All interview data, including answers and monitoring logs, must be automatically saved and securely stored to prevent data loss.

2.5.2.4. **Fairness Logging:** Proctoring violations should be logged and reviewed to ensure fairness and transparency in assessment.

2.5.2.5. **Duplicate Submission Prevention:** The system must prevent accidental multiple submissions or overlapping interview sessions to avoid conflicts or data loss.

2.5.3. Security Requirements

2.5.3.1. **Role-Based Access Control:** Only registered candidates can access interviews using JWT-based authentication. Admins and recruiters have distinct access privileges.

2.5.3.2. **Encrypted Communication:** All communication between frontend, backend, and ML microservices must use HTTPS to prevent eavesdropping.

2.5.3.3. **Data Encryption:** Sensitive candidate information, including interview responses and results, must be encrypted in MongoDB.

2.5.3.4. **Session Expiry:** Interview sessions must automatically expire after completion or periods of inactivity to prevent unauthorized access.

2.5.3.5. **Tamper-Proof Logs:** The system must maintain immutable logs of all user actions and system events for compliance and auditing purposes.

2.5.4. Usability Requirements

2.5.4.1. **Dashboard Navigation:** Candidate and admin dashboards must be easy to navigate, requiring minimal training for first-time users.

2.5.4.2. **Responsive Design:** The interface must support standard desktop resolutions and adapt for tablets where necessary.

2.5.4.3. **Proctoring Indicators:** Candidates must clearly see the interview timer, current question number, and live proctoring alerts.

2.5.4.4. **Error Messages:** All errors must be human-readable with clear instructions for corrective action.

2.5.4.5. **Admin Configuration:** Admins must easily configure violation thresholds, question sets, and reporting options.

2.5.4.6. **Accessibility:** The system should support keyboard navigation and screen readers where feasible.

2.5.4.7. **Ease of Use:** The system should prioritize simplicity over complex configuration to enable rapid adoption.

2.5.5. Reliability Requirements

2.5.5.1. **Durable Storage:** All candidate answers, proctoring logs, and reports must be reliably stored in MongoDB.

2.5.5.2. **Retry Mechanism:** The system must retry failed ML requests and queue events to prevent data loss.

2.5.5.3. **Session Resumption:** In case of network or system failure, ongoing sessions must resume without data loss.

2.5.5.4. **Accurate Reports:** All reports and analytics must accurately reflect real-time candidate activity.

2.5.5.5. **Continuous Logging:** Backend and ML microservices must continuously log events to detect anomalies and failures promptly.

2.5.6. Maintainability/Supportability Requirements

2.5.6.1. **MVC Architecture:** The backend must follow the MVC pattern, while ML services are deployed separately for independent updates.

2.5.6.2. **Documentation:** Comprehensive developer and admin manuals must be available for troubleshooting and system understanding.

2.5.6.3. **Event Logging:** All system events and errors must be logged for debugging and performance analysis.

2.5.6.4. **Microservices Updates:** The architecture must allow patching or replacement of components without system downtime.

2.5.6.5. **Monitoring Integration:** System health dashboards must monitor CPU/memory usage and ML service status in real time.

2.5.7. Portability Requirements

2.5.7.1. **Browser Compatibility:** Frontend must work on Chrome, Firefox, and Edge without requiring installation.

2.5.7.2. **Cloud Deployment:** Backend and ML microservices must be deployable on AWS, Render, Railway, or similar platforms.

2.5.7.3. **Dockerized Deployment:** All components must support Docker for consistent deployment across environments.

2.5.7.4. **OS Support:** Backend services must run on Linux or Windows servers.

2.5.8. Efficiency Requirements

2.5.8.1. **Resource Management:** ML services and backend must efficiently manage CPU and memory to handle multiple concurrent interviews.

2.5.8.2. **Proctoring Updates:** Proctoring alerts and UI updates must appear within 1–2 seconds of detection.

2.5.8.3. **Optimized Database Queries:** Database queries must be indexed and optimized to prevent performance bottlenecks.

2.5.8.4. **Webcam Frame Capture:** Candidate webcam frames should be captured every 3–5 seconds for balanced accuracy and performance.

2.5.8.5. **Frontend Memory Usage:** The frontend must maintain memory usage below 200MB on standard devices.

2.6. Domain Requirements

2.6.1. **Candidate Authentication & Access Control:** Only candidates who have successfully applied for a job and received a scheduled interview link may access the proctored session. Multi-factor authentication (e.g., email verification, OTP) ensures authorized access.

2.6.2. **Proctoring & Monitoring:** The system requires active webcam during interviews. It detects multiple screens, phones, or extra persons, triggering configurable violation events (e.g., max tab switches, face not visible count).

2.6.3. **Interview Configuration:** Organizations can define various question types, including MCQs, coding, and short questions. Each session enforces strict time limits per question or overall interview.

2.6.4. **Audit & Reporting:** Every interview maintains a detailed audit trail with timestamps, candidate actions, violations, and final decisions. Reports include visual summaries such as emotion curves, head-pose deviation, and overall score for recruiters.

2.6.5. **Data Storage & Security:** All candidate responses, proctoring logs, and reports are stored securely in MongoDB. Access is role-based, restricting sensitive data to Admin/Recruiter accounts.

2.6.6. **Reuse & Extensibility:** MediaPipe + Mini-Xception microservices are modular for future upgrades or reuse. The system allows new question types or proctoring rules to be added without major changes.

2.6.7. **Legal & Compliance:** Candidates must be informed about webcam and screen monitoring. The system complies with applicable data privacy regulations, such as GDPR for EU candidates.

Chapter 3

Use Case Analysis

Chapter 3: Use Case Analysis

This chapter delivers a detailed analysis of the functional behavior of the Smartly Hire recruitment system. It demonstrates how system users and internal services interact to accomplish key recruitment operations such as hiring automation, candidate screening, interview monitoring, violation tracking, and AI-powered report generation. The chapter establishes the functional scope, defines actor goals, maps relationship boundaries, and documents complete workflows via structured use case scenarios. This analysis ensures a shared understanding among developers, designers, testers, and stakeholders prior to entering system design and implementation phases.

3.1. Use Case Model

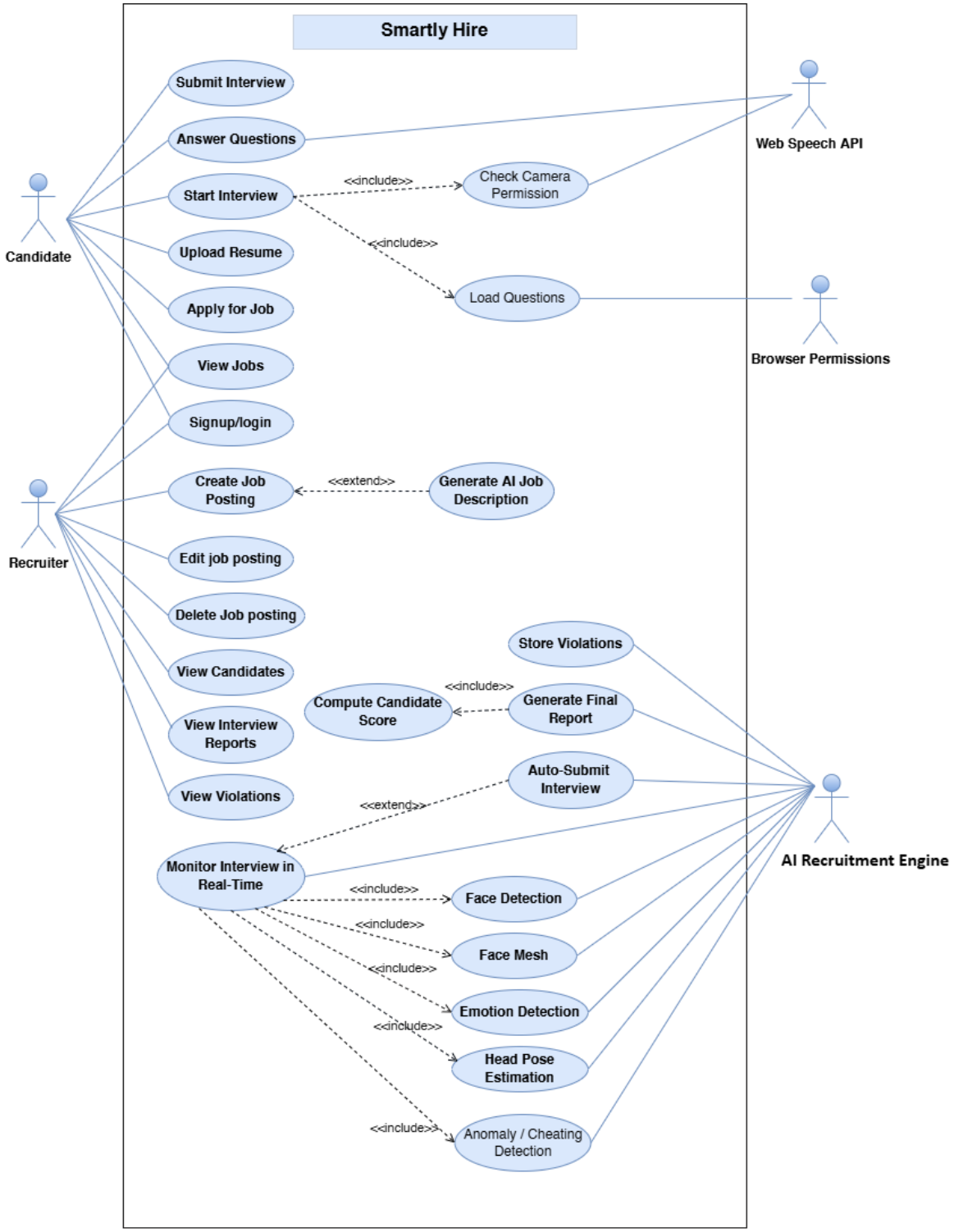
The use case model outlines the functional limits and interaction scope for the Smartly Hire system. The primary actors include:

- **Recruitment Administrator / HR Recruiter:** Responsible for job publishing, candidate management, interview reviews, violation oversight, and hiring decisions.
- **Job Candidate:** Interacts with the platform to register, explore opportunities, apply for roles, upload resumes, attend interviews, and access personal interview evaluations.

Supporting and interfacing secondary actors include:

- **Browser Permission Manager:** Handles secure access to device peripherals such as camera and screen validation.
- **AI Recruitment Engine:** The AI Recruitment Engine is responsible for real-time monitoring, emotion detection, head pose tracking, and anomaly detection during interviews. It uses computer vision and deep learning models for accurate analysis of candidate behavior [1], [2], [3], [4].

This model is represented through a structured use case diagram, ensuring coverage of actor intentions, system duties, interaction sequences, and service interdependencies for functional achievement.



3.1 Use Case (Smartly Hire)

3.2. Use Cases Description

- **Admin / Recruiter Use Cases**

UC1 (Signup/Login)

Actor: Admin

Preconditions:

1. *For Login:* Admin already has an existing registered account.
2. *For Signup:* Admin is not yet registered in the system.

Postconditions:

1. Admin is successfully authenticated.
2. Admin is redirected to the system dashboard.

Main Flow (Login Scenario):

1. Admin enters email and password.
2. System verifies if credentials match an existing record.
3. If valid, system logs in the admin.
4. Admin is granted access to the dashboard.

Alternate Flow (Incorrect Login Credentials):

1. Admin enters invalid credentials.
2. System displays an “Invalid email or password” error.
3. Admin is prompted to try again.

Main Flow (Signup Scenario):

1. Admin selects “**Create Account / Signup**” option.
2. Admin enters required information (name, email, password).
3. System checks if email already exists.
4. System creates a new admin account.
5. Admin receives confirmation and is redirected to login.

Alternate Flow (Signup Email Already Exists):

1. Admin enters an email that is already registered.
2. System displays “Email already exists” message.
3. Admin must choose a different email or proceed to login.

Business Rules:

- Password must meet security requirements.
- Email must be unique for each admin account.

UC2 (Manage Jobs)

Actor: Admin

Purpose: To view and access all job postings.

Preconditions: Admin is logged in.

Postconditions: Job list is displayed with actions.

Main Flow:

1. Admin navigates to job management.
2. System displays all available job postings.

UC3 (Create Job Posting)

Actor: Admin

Preconditions: Admin is logged in.

Postconditions: New job posting is created.

Main Flow:

1. Admin enters job details.
2. Admin optionally triggers AI job description generation (extends UC6).
3. System saves the job.

   Extension:

   UC6 Generate AI Description extends this use case.

UC4: Edit Job Posting

Actor: Admin

Preconditions: Job must exist.

Postconditions: Updated job details saved.

Main Flow:

1. Admin selects existing job.
2. Admin updates fields.
3. System saves changes.

UC5: Delete Job Posting

Actor: Admin

Preconditions: Job must exist.

Postconditions: Job is removed.

Main Flow:

1. Admin selects job.
2. Admin confirms deletion.
3. System deletes job.

UC6: Generate AI Job Description *(Extend)*

Actor: Admin

Preconditions: Admin is creating/editing a job posting.

Postconditions: AI-generated job description shown.

Main Flow:

1. Admin clicks “Generate Description”.
2. System creates AI-based job description.
3. Admin accepts or edits it.

UC7: View Candidates

Actor: Admin

Purpose: To see list of applicants for a job.

Preconditions: Candidates must have applied.

Postconditions: Candidate list displayed.

UC8: View Interview Reports

Actor: Admin

Purpose: To analyze AI-generated reports.

Preconditions: Candidate must have completed an interview.

Postconditions: Report shown.

UC9: View Violations

Actor: Admin

Purpose: To review cheating/violation logs.

Preconditions: System recorded violations.

Postconditions: Violation details displayed.

Candidate Use Cases

UC10: login/Signup

Primary Actor: Candidate

Preconditions:

1. ***For Signup:*** Candidate does not have an existing registered account.
2. ***For Login:*** Candidate must already be registered.

Postconditions:

1. Candidate is authenticated successfully.
2. Candidate is redirected to the job dashboard or homepage.

Main Flow (Signup Scenario):

1. Candidate selects **"Signup / Create Account."**
2. Candidate enters required details (name, email, password).

3. System checks if email is already registered.
4. If not registered, system creates a new candidate account.
5. Candidate receives confirmation and is redirected to login.

Alternate Flow (Signup -> Email Already Exists):

1. Candidate enters an email that already exists in the system.
2. System displays message: **"Account already exists."**
3. Candidate is prompted to log in instead.

Main Flow (Login Scenario):

1. Candidate enters email and password.
2. System validates credentials.
3. If valid, system logs in the candidate.
4. Candidate is redirected to the job dashboard.

Alternate Flow (Invalid Credentials):

1. Candidate enters incorrect email or password.
2. System displays: **"Invalid credentials. Please try again."**
3. Candidate retries login.

Business Rules:

- Email must be unique for each candidate.
- Password must meet security requirements.

UC11: View Jobs

Actor: Candidate

Main Flow: System displays available jobs.

UC12: Apply for Job

Actor: Candidate

Preconditions: Candidate is logged in.

Postconditions: Application submitted.

UC13: Upload Resume

Actor: Candidate

Preconditions: Candidate is applying for a job.

Postconditions: Resume uploaded.

UC14: Start Interview

Actor: Candidate

Preconditions: Candidate applied and has permission.

Postconditions: Interview begins.

Includes: UC15, UC16, UC17

Main Flow:

1. Candidate clicks "*Start Interview*".
2. System checks permissions (camera).
3. System loads questions.
4. Interview session starts.

UC15: Check Camera Permission *(Include)*

Actor: Browser Permissions

Purpose: Ensure camera access.

Main Flow: Browser prompts user to allow camera.

UC16: Load Questions *(Include)*

Actor: Web Speech API (partial) / System

Purpose: Load question list and prepare text-to-speech.

UC17: Answer Questions

Actor: Candidate

Secondary Actors: Web Speech API

Postconditions: Responses stored.

UC18: Submit Interview

Actor: Candidate

Purpose: End interview session.

Postconditions: Submission stored.

AI Recruitment & Monitoring Use Cases

UC19: Monitor Interview in Real-Time

Actor: AI Recruitment Engine

Purpose: Continuous monitoring of candidate behavior in real time.

Includes: UC21, UC22, UC23

Postconditions: Monitoring data generated and streamed for analysis.

UC20: Detect Candidate Emotion (Include)

Actor: AI Recruitment Engine

Model: Mini-Xception CNN

Purpose: Categorize emotional state for interview insights

Postconditions: Emotion state logged.

UC21: Head Pose & Attention Tracking (Include)

Actor: AI Recruitment Engine

Model: MediaPipe + SolvePnP

Purpose: Assess candidate attention and head movement.

UC22: Anomaly & Cheating Detection (Include)

Actor: AI Recruitment Engine

Checks: Phone detection, additional person, face not visible, multi-screen usage

Purpose: Flag behavioral anomalies and recruitment violations.

UC23: Record Violations

Actor: AI Recruitment Engine

Purpose: Store violation logs securely.

Postconditions: Violations stored for admin review.

UC24: Auto-Submit Interview *(Extend)*

Actor: AI Recruitment Engine

Triggered When: Predefined violation limits exceeded

Extends: UC20 Live Monitoring

Postconditions: Interview auto-submitted with candidate notification.

UC25: Create Interview Evaluation Report

Actor: AI Recruitment Engine

Purpose: Generate final interview analysis document.

Includes: UC27

UC26: Compute Candidate Hiring Score (Include)

Actor: AI Recruitment Engine

Purpose: Calculate cumulative performance score considering responses, emotions, attention, and violations.

Chapter 4

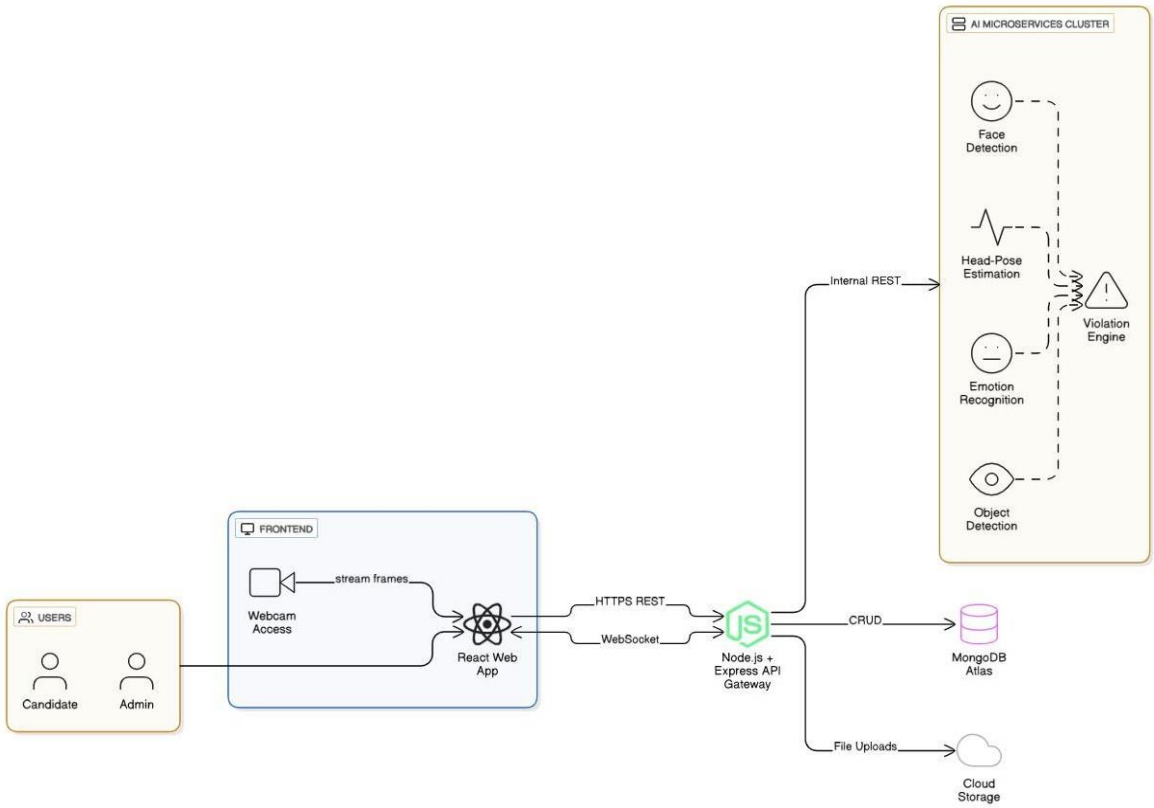
System Design

Chapter 4: System Design

This chapter presents the detailed system design of the Smartly Hire platform by modeling both structural and behavioral components of the recruitment system. It explains the overall architecture, data entities, object relationships, process interactions, and AI-driven proctoring services integrated using modern machine learning frameworks [5]. The design ensures scalability, modularity, and efficient real-time processing for AI-based monitoring systems.

4.1. Architecture Diagram

The architecture diagram presents the system design and service interaction for Smartly Hire, an AI-powered recruitment and interview platform. It highlights secure user access, scalable frontend hosting, API integration, and real-time AI proctoring workflows. Interview data, integrity events, and evaluation reports are stored in structured databases and object storage. AI question generation and answer assessment are orchestrated through LLM inference services, supported by monitoring and CI/CD pipelines.



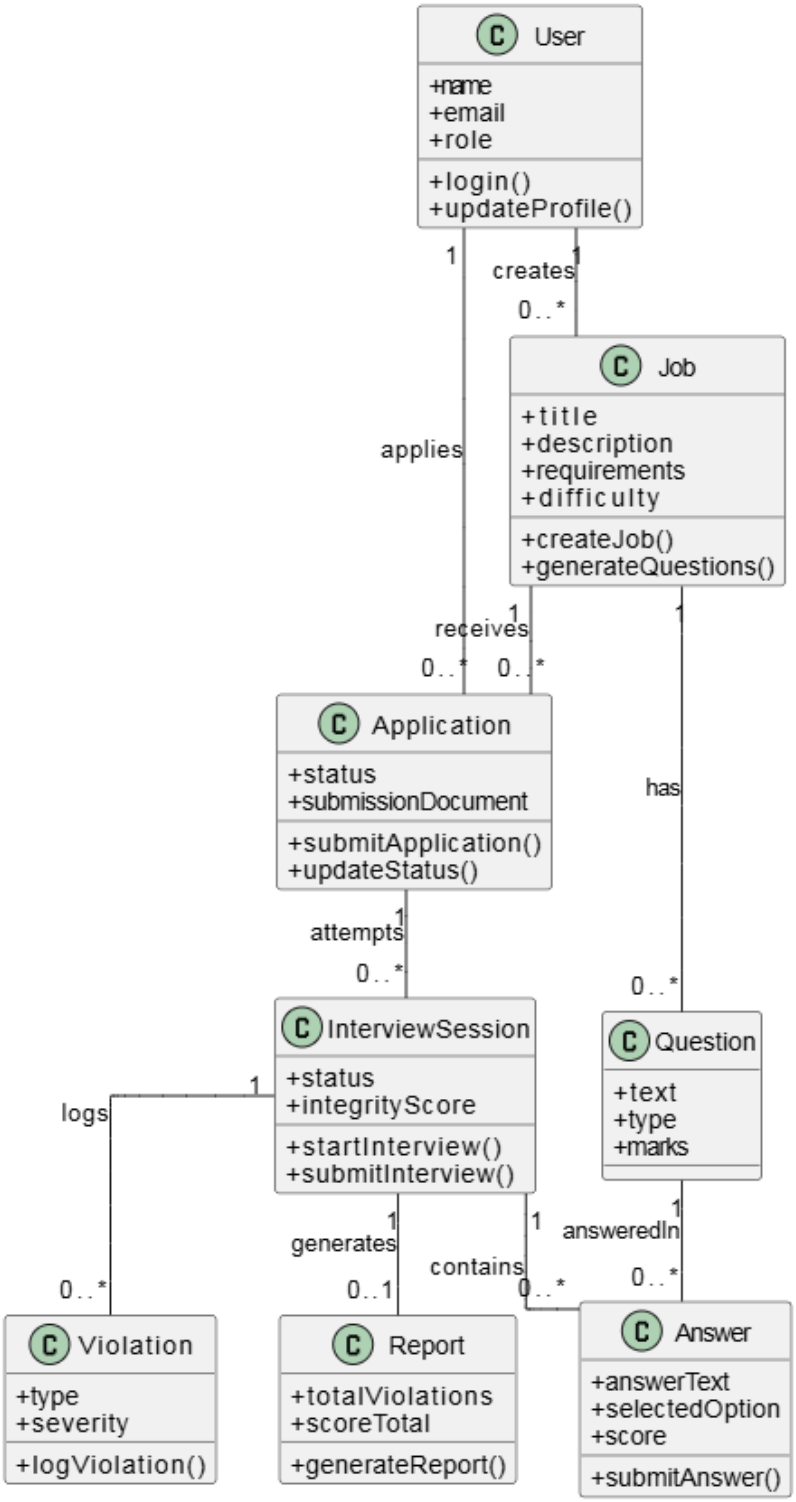
4.1 Architecture diagram

4.2. Domain Model

The domain model of Smartly Hire illustrates the **conceptual entities** and their relationships within the recruitment system.

It focuses on the **real-world objects** such as users, jobs, applications, interviews, and reports.

Only **business-relevant attributes and high-level behaviors** are included to show system understanding.



5.2 Domain Model diagram

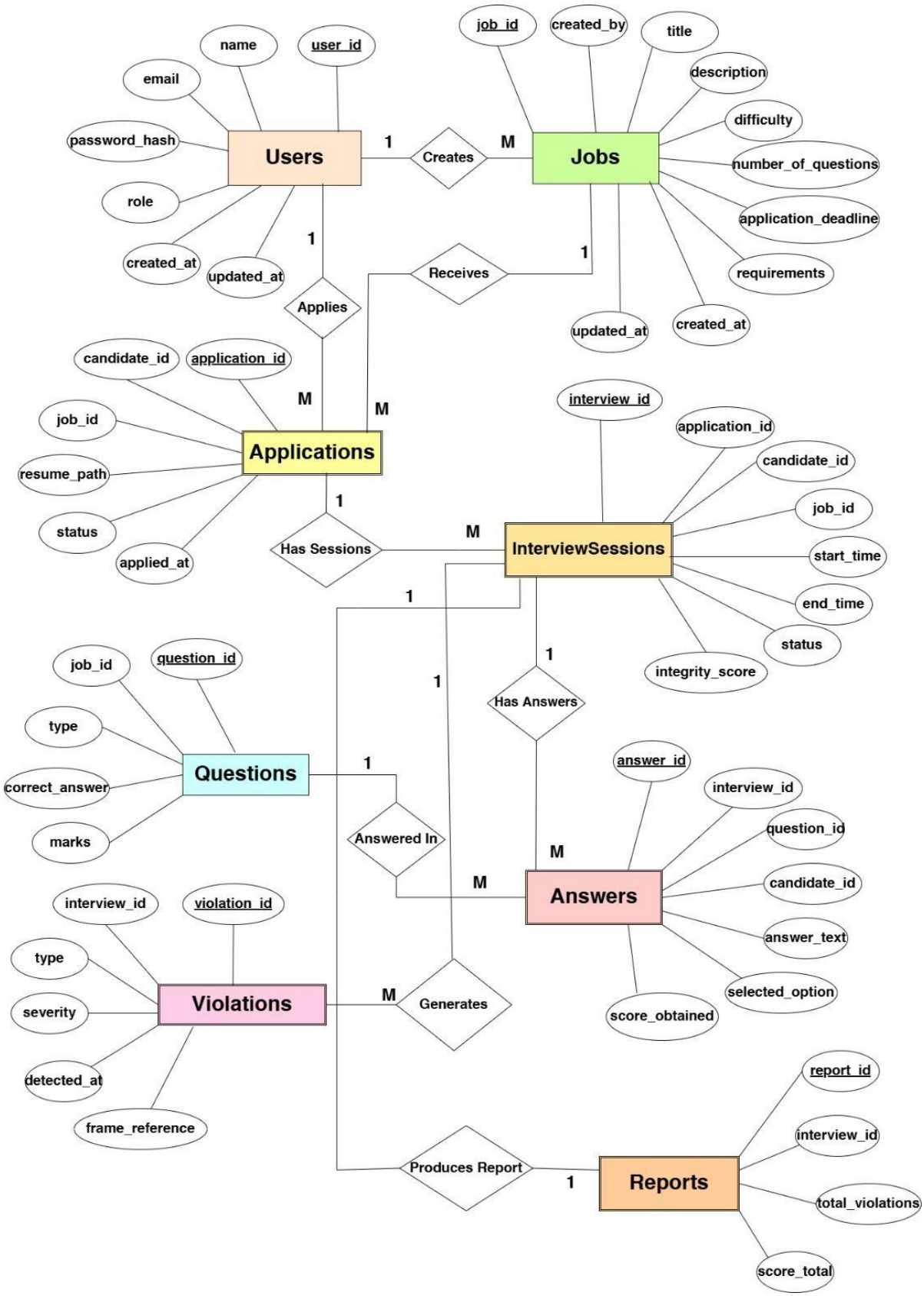
4.3. Entity Relationship Diagram with data dictionary

The ER diagram for **Smartly Hire** visualizes the complete recruitment and interview integrity data model.

It maps key entities and relationships to ensure seamless storage of job posts, interview responses, and real-time proctoring violations.

The ERD establishes cardinality (1:M, 1:1), foreign key linkage, and data flow dependencies critical for system consistency.

This structured representation serves as the foundation for database design, evaluation scoring, and integrity monitoring across interviews.



6.3 Entity Relationship Diagram

Data Dictionary:

Entity: users

4.1 Entity: users

Attribute	Type	Size	Key	Constraints	Description
user_id	ObjectId	24	PK	Not Null, Unique	Unique identifier for each user in the system
Name	String	100		Not Null	Full name of the Admin or Candidate
Email	String	255		Not Null, Unique	Email used for login and communication
password_hash	String	255		Not Null	Secure hashed password for authentication
Role	String	20		Enum(admin, candidate)	Defines system access level
created_at	DateTime			Auto Generated	Timestamp when account was created
updated_at	DateTime			Auto Generated	Timestamp of last update to the user

Entity: Jobs

4.2 Entity: Jobs

Attribute	Type	Size	Key	Constraints	Description
job_id	ObjectId	24	PK	Not Null, Unique	Unique identifier for each job posting
Title	String	150		Not Null	Job position title
Description	Text			Not Null	Full job details and expectations
Requirements	Array<String>				List of required skills/qualifications
Difficulty	String	20		Enum(easy, medium, hard)	Difficulty level for interview questions
number_of_questions	Integer				Total questions generated for interview
application_deadline	DateTime				Last date/time to apply for job
created_by	ObjectId	24	FK	Not Null	References the admin user who created the job
created_at	DateTime			Auto Generated	Job creation timestamp
updated_at	DateTime			Auto Generated	Job last modified timestamp

Entity: Applications

4.3 Entity: Applications

Attribute	Type	Size	Key	Constraints	Description
application_id	ObjectId	24	PK	Not Null, Unique	Unique identifier for job application
candidate_id	ObjectId	24	FK	Not Null	References candidate user who applied
job_id	ObjectId	24	FK	Not Null	References job being applied for
resume_path	String	255			File path or storage reference to uploaded resume
status	String	20		Enum (applied, invited, completed)	Current application stage
applied_at	DateTime			Auto Generated	Time when candidate applied for job

Entity: Interview Sessions:

4.4 Entity: Interview Sessions

Attribute	Type	Size	Key	Constraints	Description
interview_id	ObjectId	24	PK	Not Null, Unique	Unique interview attempt/session id
application_id	ObjectId	24	FK	Not Null	References linked application
candidate_id	ObjectId	24	FK	Not Null	Interview attempting candidate
job_id	ObjectId	24	FK	Not Null	References job being interviewed for
start_time	DateTime			Not Null	Interview start time
end_time	DateTime			Not Null	Interview end time
status	String	20		Enum(started, submitted, evaluated)	Interview progress state
integrity_score	Integer			Min 0, Max 100	Final computed integrity score from proctoring
created_at	DateTime			Auto Generated	Session creation timestamp

Entity: Questions:

4.5 Entity: Questions

Attribute	Type	Size	Key	Constraints	Description
question_id	ObjectId	24	PK	Not Null, Unique	Unique question identifier
job_id	ObjectId	24	FK	Not Null	References parent job
type	String	20		Enum(mcq, short, long, coding)	Type of interview question
text	Text	Not Null		Not Null	Actual question prompt text
options_json	JSON				MCQ options (only if type=mcq)
correct_answer	String				Optional golden answer for auto-eval
marks	Integer			Not Null	Maximum score for this question
created_at	DateTime			Auto Generated	Question creation time

Entity: Answers:

4.6 Entity: Answers

Attribute	**Type**	**Size**	**Key**	**Constraints**	**Description**
answer_id	ObjectId	24	PK	Not Null, Unique	Unique identifier for submitted answer
interview_id	ObjectId	24	FK	Not Null	References interview session
question_id	ObjectId	24	FK	Not Null	References answered question
candidate_id	ObjectId	24	FK	Not Null	Candidate who gave answer
answer_text	Text				Textual/coding answer if applicable
selected_option	String				Option chosen for MCQ questions
score_obtained	Integer				Score given after evaluation
evaluated_by	String	10		Enum(ai, manual, null)	Evaluation source
evaluated_at	DateTime				Evaluation timestamp

Entity: Violations:

4.7 Entity: Violations

Attribute	Type	Size	Key	Constraints	Description
violation_id	ObjectId	24	PK	Not Null, Unique	Unique violation identifier
interview_id	ObjectId	24	FK	Not Null	References interview session
type	String	30		Enum(no_face, multiple_faces, looking_away, phone, book, paper, other)	Category of integrity violation
severity	String	10		Enum(low, medium, high)	Severity level of the violation
detected_at	DateTime			Auto Generated	Time when incident was detected
frame_reference	String	255			Video frame/snapshot path stored externally
metadata	JSON				Additional details like bounding box, confidence

Entity: Reports:

4.8 Entity: Reports

Attribute	Type	Size	Key	Constraints	Description
report_id	ObjectId	24	PK	Not Null, Unique	Unique interview report identifier
interview_id	ObjectId	24	FK	Not Null, Unique (1:1)	Links to evaluated interview session
total_violations	Integer			Not Null	Count of integrity violations in session
score_total	Integer				Final interview evaluation score
report_json_path	String	255			Path to stored JSON evaluation result
report_pdf_path	String	255			Optional path to generated PDF report
created_at	DateTime			Auto Generated	When report was created

Relationships (Data Dictionary Linkage):

4.9 Relationships (Data Dictionary Linkage)

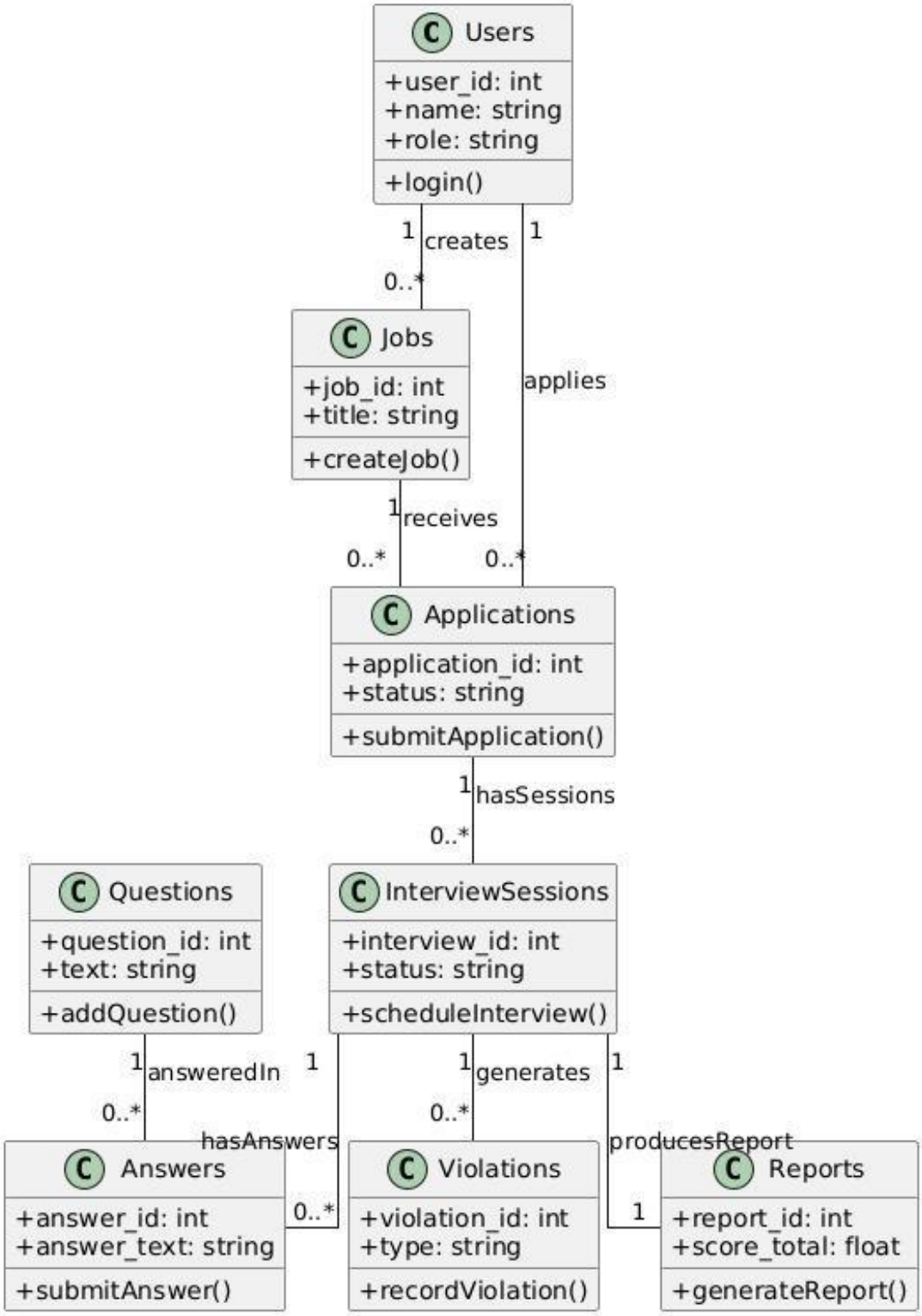
From	To	Cardinality	Description
Users.user_id	Jobs.created_by	1:M	One admin can create many jobs
Users.user_id	Applications.candidate_id	1:M	One candidate can apply to many jobs
Jobs.job_id	Applications.job_id	1:M	One job can receive many applications
Applications.application_id	InterviewSessions.application_id	1:M	One application may allow multiple interview attempts (default supports retries)
InterviewSessions.interview_id	Answers.interview_id	1:M	One interview session has many answers
Questions.question_id	Answers.question_id	1:M	One question can be answered in many interview sessions
InterviewSessions.interview_id	Violations.interview_id	1:M	One interview may log many violation events
InterviewSessions.interview_id	Reports.interview_id	1:1 (0/1 initially, then 1)	One interview has one report after evaluation

4.4. Class Diagram

The class diagram of Smartly Hire represents the **implementation-level structure** of the system.

It includes **detailed attributes, methods, and visibility** for each class, reflecting how the system will be built.

Relationships between classes, along with **multiplicities and ownership types**, are clearly defined. This diagram

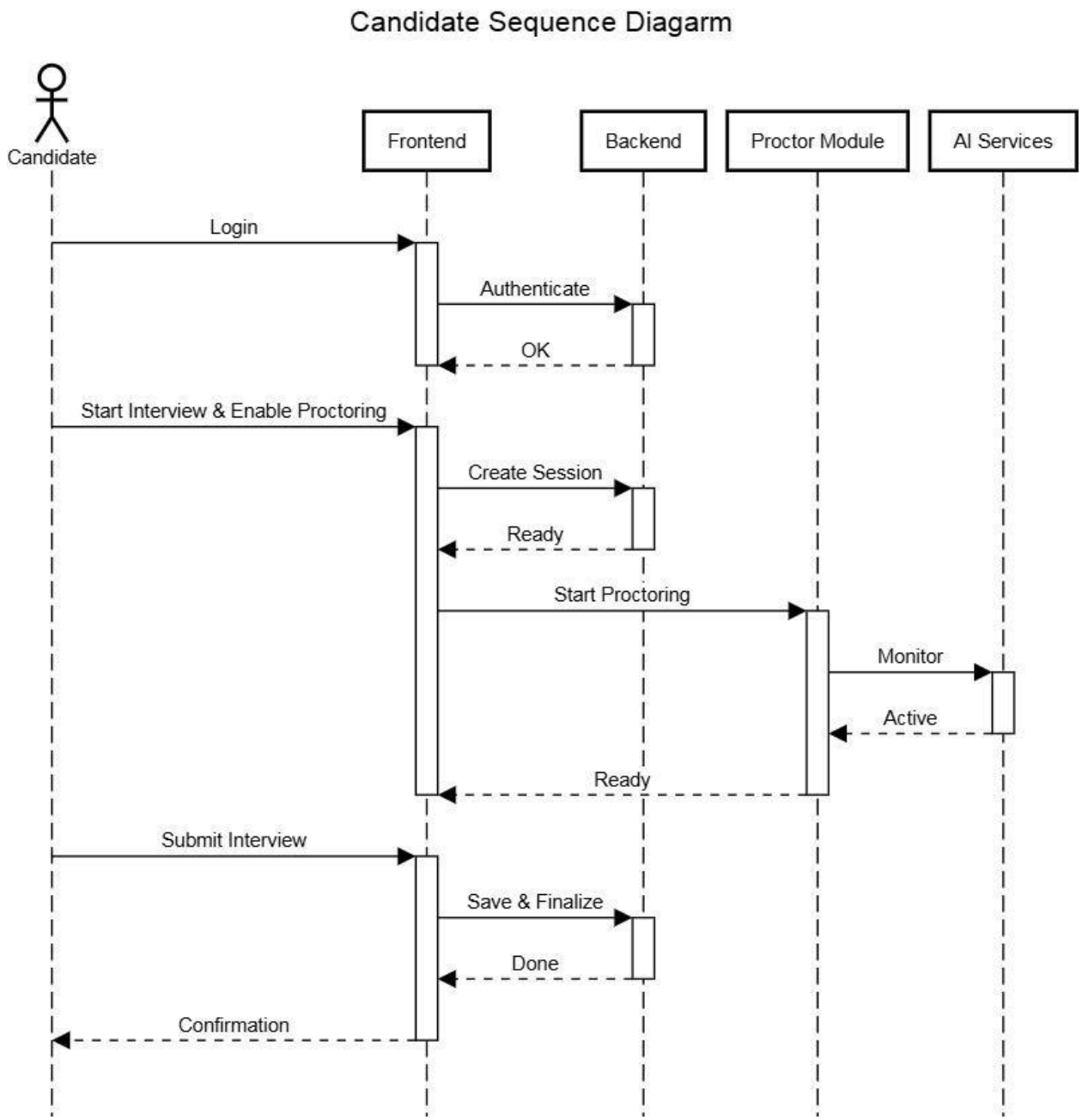


serves as a **technical blueprint** for developers and guides the actual software implementation.

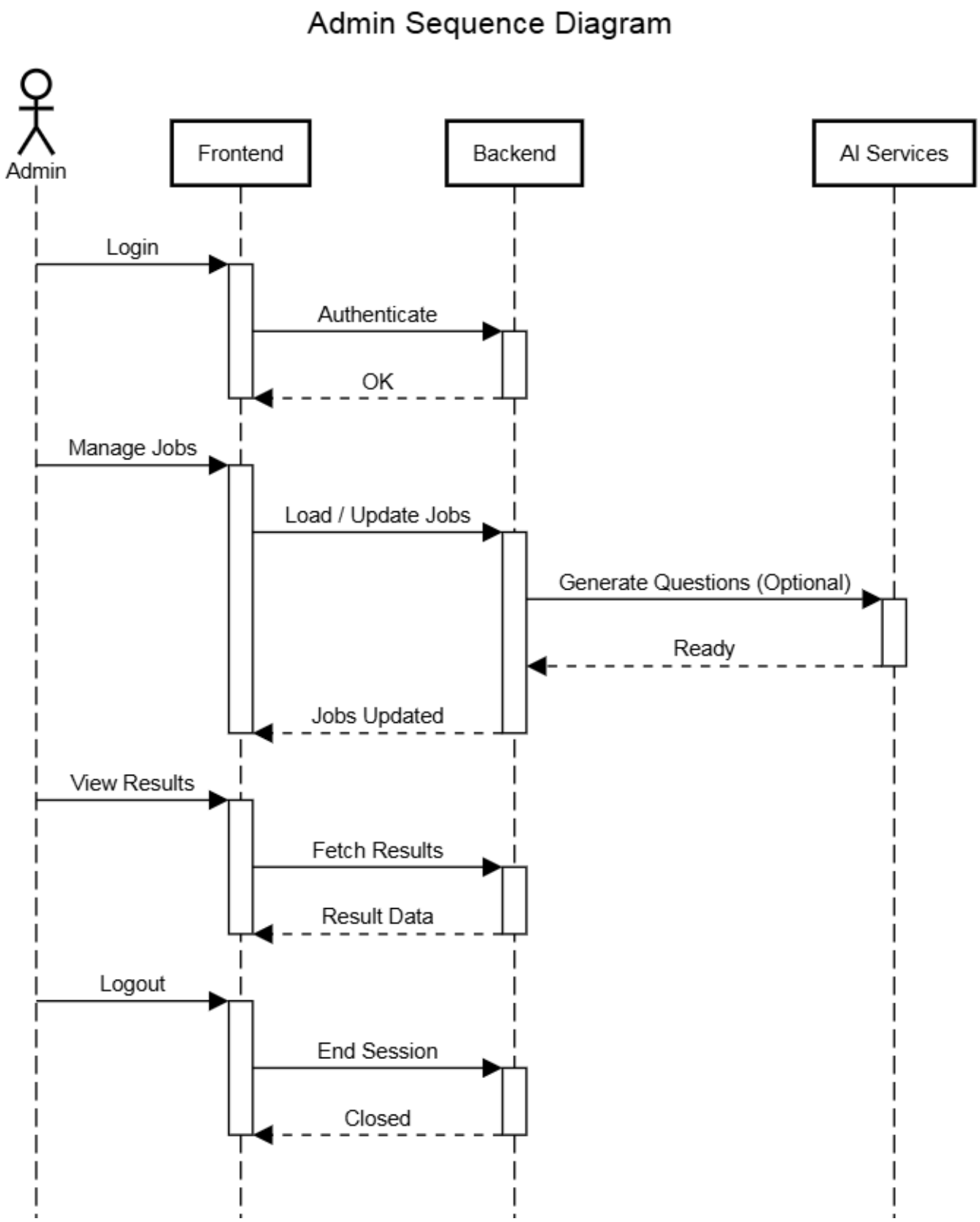
7.4 Class Diagram

4.5. Sequence Diagram

The Sequence Diagrams illustrate the dynamic interactions between users and system components in Smartly Hire. It shows the chronological flow of messages between actors, the frontend, backend controllers, database, and AI/ML services. The diagram captures both candidate interview workflows, including live proctoring and answer evaluation, and admin job creation processes. By visualizing these interactions, it provides a clear understanding of the system's runtime behavior and message sequence.



8.5 Sequence Diagram for Candidate



9.6 Sequence Diagram for Admin

4.6. Operation contracts

Operation Contracts illustrate the **detailed specifications of system operations** in Smartly Hire. They describe the **preconditions, postconditions, inputs, outputs, and responsibilities** of each major function.

These contracts provide a **clear understanding of how each operation affects the system state** and interacts with other entities.

They serve as a **bridge between use case analysis and class design**, ensuring consistent and precise implementation.

User Operations:

4.10 User Operations

Operation Name	**User.login(email, password)**
Cross-Reference	User Authentication Use Case
Precondition	User exists in the system
Postcondition	User session initiated if credentials are valid
Inputs	email : String, password : String
Outputs	success : Boolean
Responsibility	Authenticates a user and starts a session
Operation Name	**User.updateProfile(name, email)**
Cross-Reference	Update Profile Use Case
Precondition	User is logged in
Postcondition	User's name and email updated in the system
Inputs	name : String, email : String
Outputs	updated : Boolean
Responsibility	Updates user profile information

Job Operations:

4.11 Job Operations (User Authentication)

Operation Name	**Job.createJob()**
Cross-Reference	User Authentication Use Case
Precondition	User exists in the system
Postcondition	User session initiated if credentials are valid
Inputs	email : String, password : String
Outputs	success : Boolean
Responsibility	Authenticates a user and starts a session
Operation Name	**User.updateProfile(name, email)**
Cross-Reference	Update Profile Use Case
Precondition	User is logged in
Postcondition	User's name and email updated in the system
Inputs	name : String, email : String
Outputs	updated : Boolean
Responsibility	Updates user profile information

Job Operations:

4.12 Job Operations (Admin)

Operation Name	**Job.createJob()**
Cross-Reference	Admin creates job Use Case
Precondition	User role = "admin"
Postcondition	Job saved in system, available for candidates
Inputs	title : String, description : String, requirements : List<String>, difficulty : String
Outputs	jobId : ObjectId

Responsibility	Creates a new job posting
Operation Name	**Job.generateQuestions(count)**
Cross-Reference	Question Generation Use Case
Precondition	Job exists
Postcondition	List of interview questions generated and returned
Inputs	count : Integer
Outputs	questions : List<Question>
Responsibility	Generates interview questions using AI or template

Application Operations:

4.13 Application Operations

Operation Name	**Application.updateStatus(status)**
Cross-Reference	Application Status Update
Precondition	Application exists
Postcondition	Application.status updated
Inputs	status : String (applied/invited/completed)
Outputs	updated : Boolean
Responsibility	Updates the status of an application
Operation Name	**Application.submitApplication(submissionDocument)**
Cross-Reference	Candidate applies for Job Use Case
Precondition	User role = "candidate", Job deadline not passed
Postcondition	Application linked to user and job, status = "submitted"
Inputs	submissionDocument : String
Outputs	confirmation : Boolean
Responsibility	Submits a job application along with document

InterviewSession Operations:

4.14 InterviewSession Operations

Operation Name	**InterviewSession.startInterview()**
Cross-Reference	Start Interview Use Case
Precondition	Candidate has submitted application
Postcondition	Interview status = "started", timer begins, proctoring activated
Inputs	None
Outputs	sessionActive : Boolean
Responsibility	Begins the interview session
Operation Name	**InterviewSession.submitInterview()**
Cross-Reference	Submit Interview Use Case
Precondition	Interview in progress
Postcondition	Interview status = "submitted", answers stored, integrity score logged
Inputs	None
Outputs	submissionConfirmed : Boolean
Responsibility	Submits the completed interview for evaluation

Question & Answer Operations:

4.15 Question & Answer Operations

Operation Name	**Answer.submitAnswer(answerText, selectedOption)**
Cross-Reference	Candidate Answers Question
Precondition	Interview in progress

Postcondition	Answer stored and linked to interview and question
Inputs	answerText : String, selectedOption : String
Outputs	submitted : Boolean
Responsibility	Records candidate's answer to a question
Operation Name	**Question.evaluateAnswer(answer)**
Cross-Reference	AI/Manual Evaluation Use Case
Precondition	Answer exists
Postcondition	Score calculated and returned
Inputs	answer : Answer
Outputs	score : Integer
Responsibility	Evaluates an answer and returns the score

Violation Operations:

4.16 Violation Operations

Operation Name	**Violation.logViolation(type, severity)**
Cross-Reference	Proctoring Detection Use Case
Precondition	Interview is active, candidate under observation
Postcondition	Violation logged with timestamp and type
Inputs	type : String (no_face, phone, book, etc), severity : String (low/medium/high)
Outputs	violationLogged : Boolean
Responsibility	Logs a proctoring violation during the interview

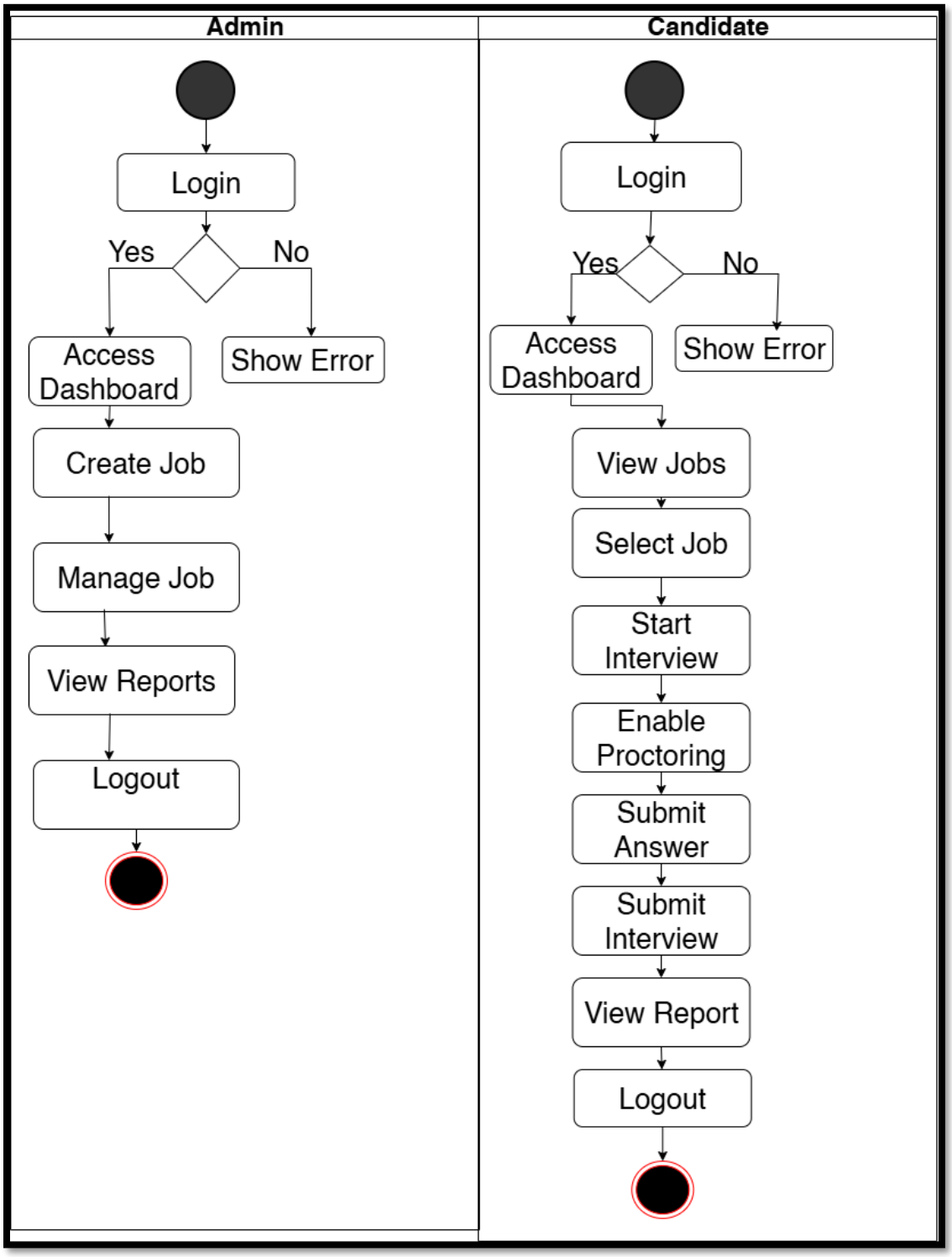
Report Operations:

4.17 Report Operations

Operation Name	**Report.generateReport()**
Cross-Reference	Evaluation & Reporting Use Case
Precondition	Interview submitted and evaluated
Postcondition	Report generated with total score, violations, and optional PDF/JSON
Inputs	None
Outputs	reportPath : String
Responsibility	Produces final interview report including scores and proctoring summary

4.7. Activity Diagram

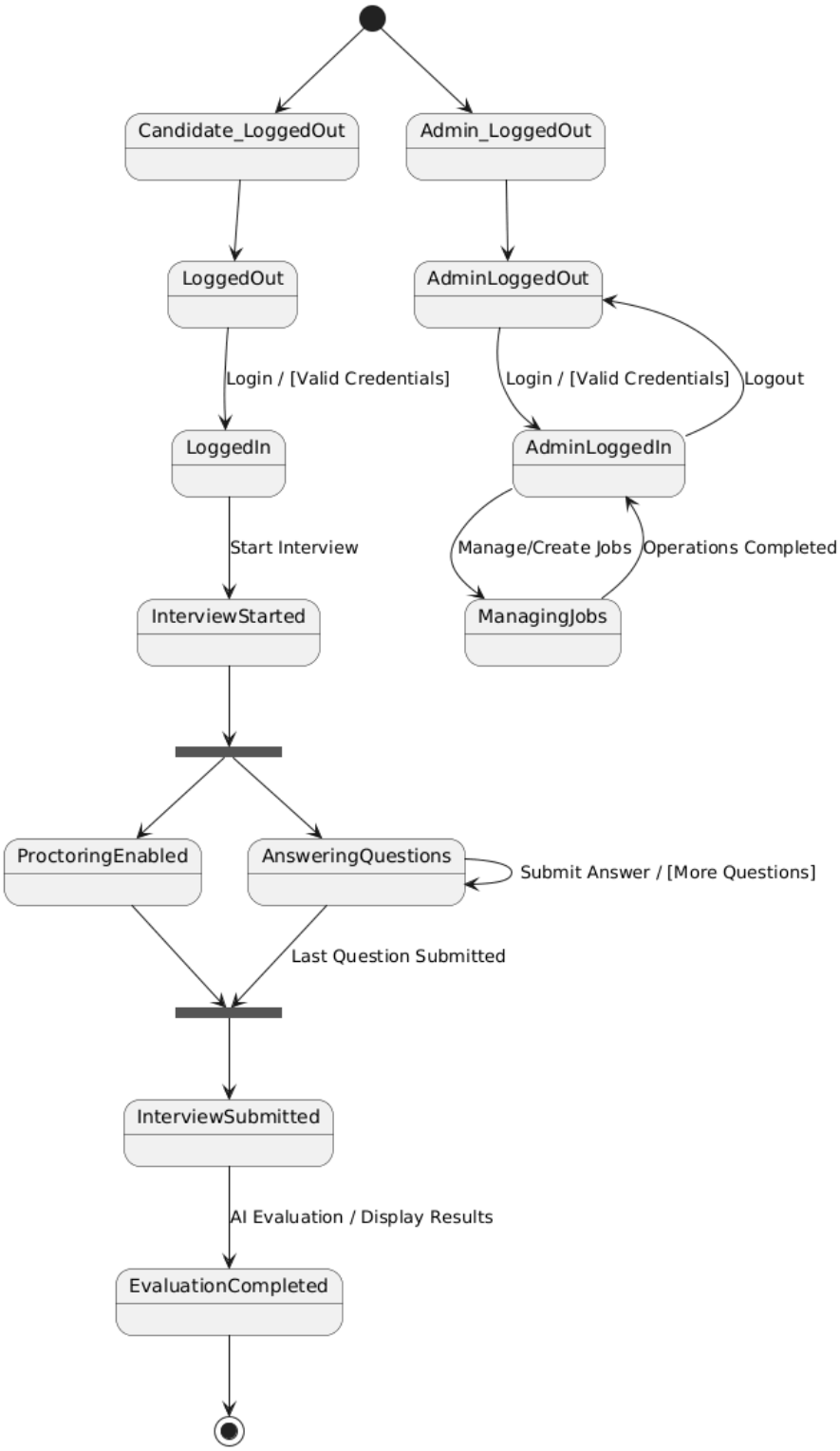
The activity diagram illustrates the **end-to-end workflows of the Smartly Hire platform**, highlighting the interactions between **Admin, Candidate, and AI/Proctoring modules**. It captures key activities such as login, job creation, interview initiation, live proctoring, question answering, and optional AI evaluation. Decision points, loops, and concurrent processes, like camera permission checks and question loading, are clearly represented to show dynamic control flow. Swimlanes are used to group activities by actor, providing clarity on role-specific responsibilities. Overall, the diagram visually communicates the sequence, conditions, and concurrency of tasks within the recruitment system.



10.7 Activity Diagram

4.8. State Transition Diagram

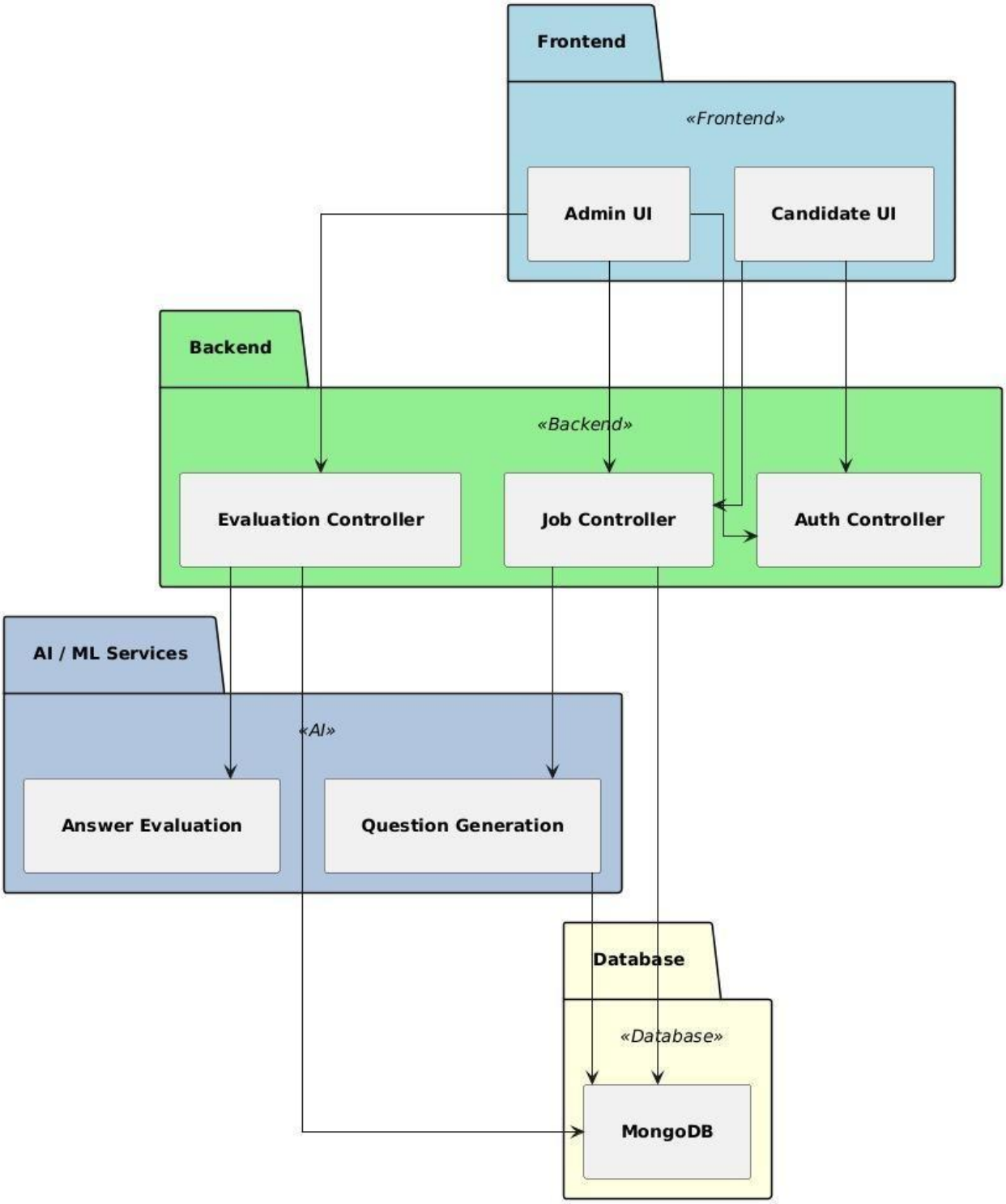
The **State Transition Diagram** illustrates the dynamic behavior of the Smartly Hire system, capturing the various states a user can pass through during job application and interview processes. It highlights both candidate and admin workflows, including login, job browsing, application submission, interview taking, proctoring, evaluation, and report generation. The diagram also models continuous proctoring events and a dynamic loop for answering any number of questions, ensuring system integrity. This visual representation provides a clear and structured overview of the system's runtime behavior and interactions.



11.8 State Transition Diagram

4.9. Component Diagram

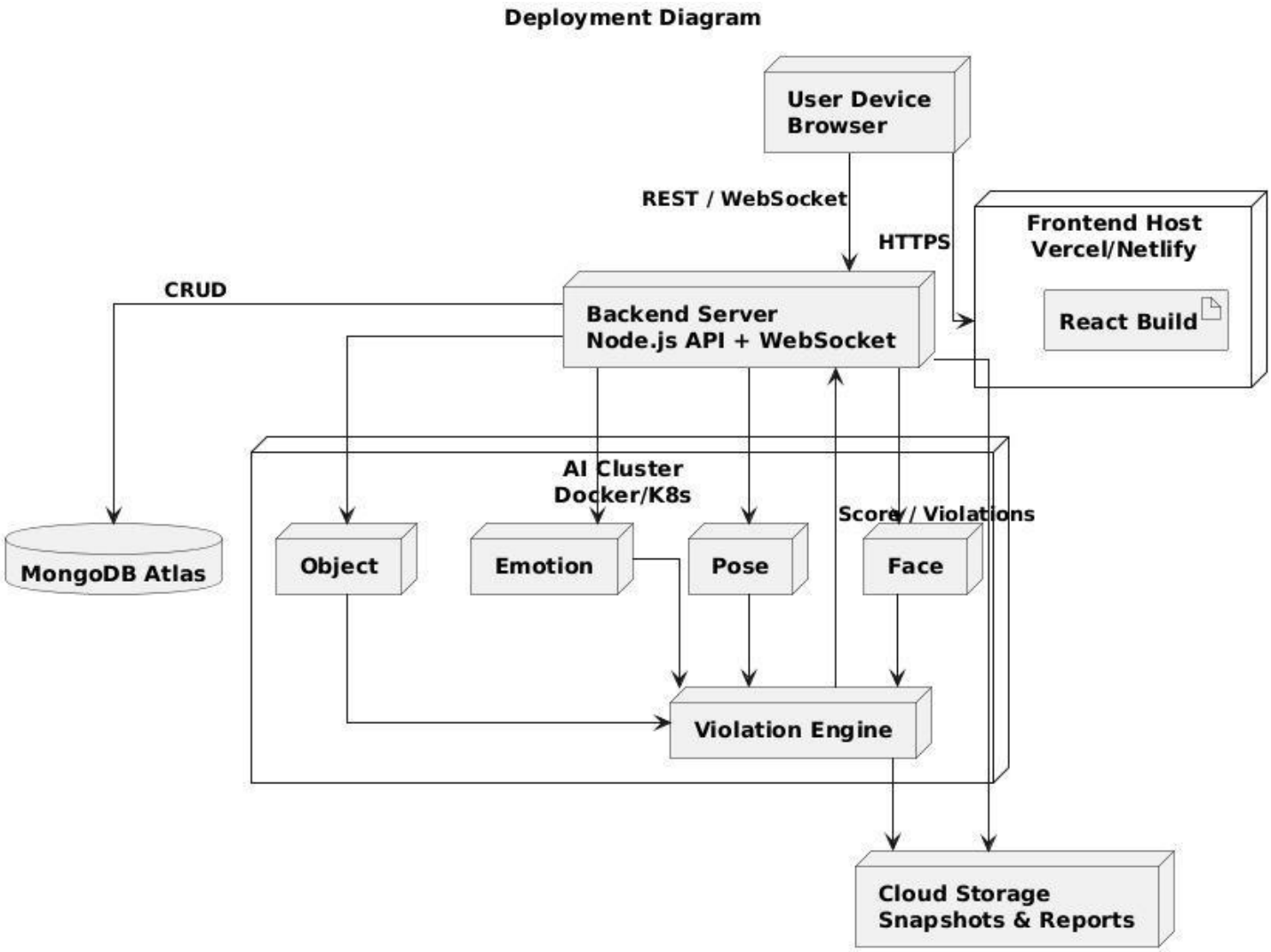
The Component Diagram illustrates the high-level architecture of the Smartly Hire system, showing how the system is modularized into distinct components. It highlights the separation between frontend, backend, database, and AI/ML services, along with the interfaces and dependencies between them. The diagram emphasizes the modularity, reusability, and communication paths among components, ensuring clarity in system design. This visual representation provides a structured overview of the system's architecture for both development and documentation purposes.



12.9 Component Diagram

4.10. Deployment Diagram

The Deployment Diagram illustrates the physical architecture of the Smartly Hire system, showing how software components are deployed on hardware nodes. It highlights the mapping of the frontend, backend, database, and AI/ML services to specific servers and devices. The diagram emphasizes communication paths, protocols, and dependencies between nodes, ensuring clarity in system deployment. This visual representation provides a clear overview of the system's infrastructure for development, testing, and production environments.



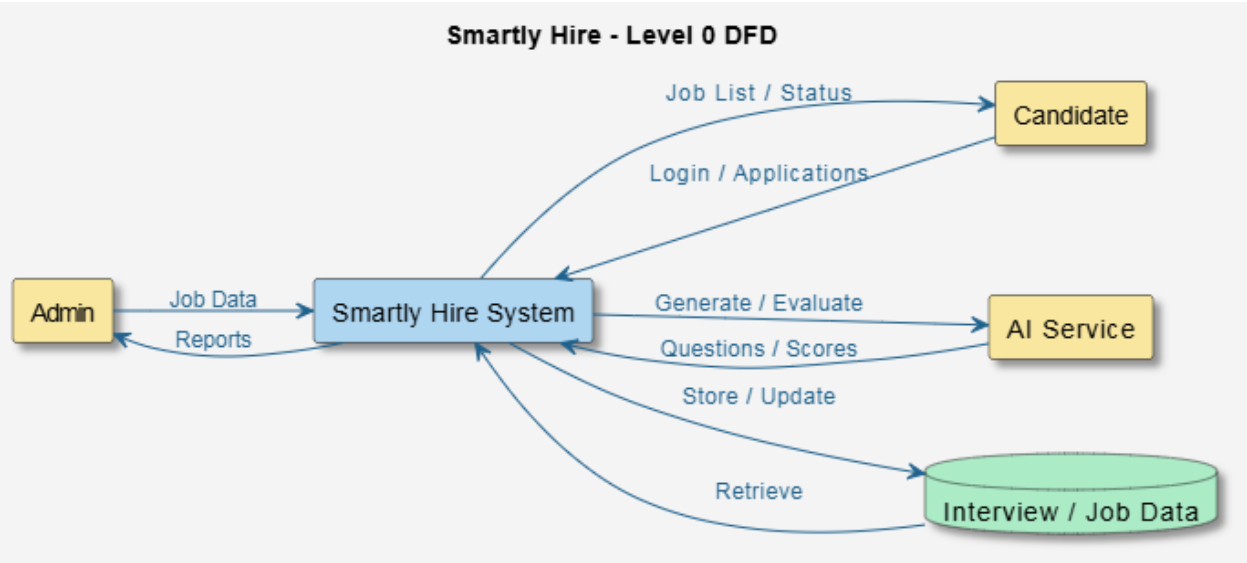
13.10 Deployment Diagram

4.11. Data Flow diagram

The Data Flow Diagram (DFD) illustrates how data moves through the Smartly Hire system, showing processes, data stores, and interactions with external entities. It provides a clear visual representation of system functionality, supporting analysis of data flow and integration of AI-based evaluation and proctoring. These diagrams help stakeholders understand the flow of information and the key operations performed by the system. Overall, DFDs enhance system clarity and support efficient system design and communication.

Level 0 DFD:

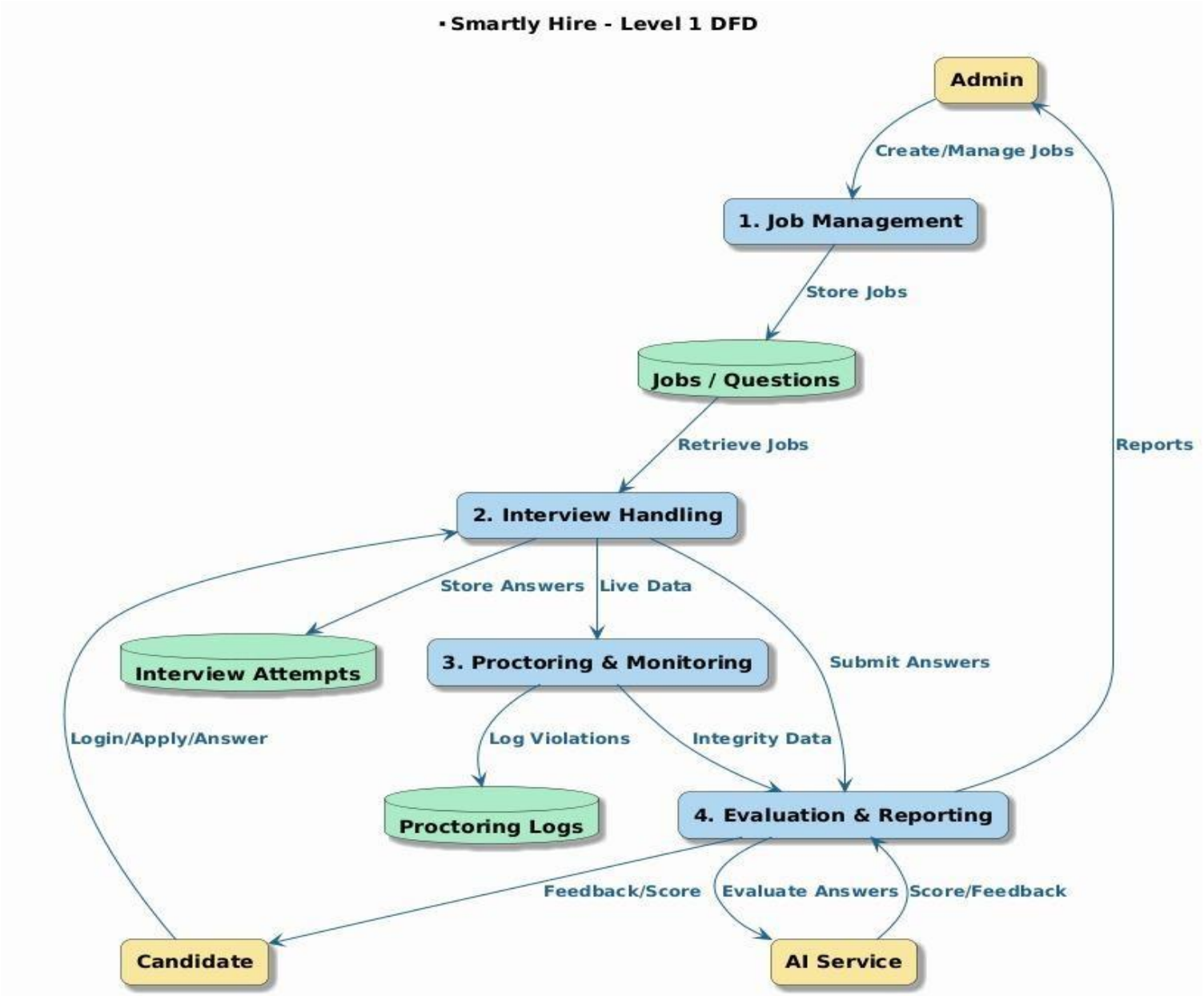
Level 0 provides a high-level view of the system as a single process interacting with external entities like Admin, Candidate, and AI Services, showing the main inputs and outputs.



14.11 Level 0 DFD

Level 1 DFD:

Level 1 decomposes the main process into sub-processes including Job Management, Interview Handling, Proctoring & Monitoring, and Evaluation & Reporting, showing detailed data flows and storage interactions.



15.12 Level 1 DFD

Chapter 5

Implementation

Chapter 5: Implementation

This chapter outlines the technical realization of the **Smartly Hire** recruitment platform. It covers system workflow, module integration, core components, libraries, security implementation, deployment pipelines, tool usage, coding standards, and version control practices followed throughout development. The implementation demonstrates how AI-driven monitoring, automated screening, job processing, and interview assessment are unified to create an intelligent digital hiring ecosystem.

The system is built using a microservices-based architecture integrating modern web and AI technologies including React, Node.js, MongoDB, and Python-based ML services for real-time proctoring and evaluation capabilities [5], [12], [3].

5.1. Important Flow Control/Pseudo codes

The following pseudocode reflects the primary control logic implemented within the system, spanning administrator authentication, job publishing, candidate application processing, and AI-based interview monitoring and evaluation.

Admin Login Flow (Pseudocode):

This flow verifies the credentials of the admin and ensures that only authorized users can access the admin dashboard. It generates a JWT token upon successful authentication for secure session management using standard authentication practices [5].

```
//Admin Login Flow
FUNCTION AdminLogin(email, password):
    user = FindUserByEmail(email)

    IF user DOES NOT EXIST:
        RETURN 'Invalid Credentials'

    IF user.role != 'admin':
        RETURN 'Unauthorized Access'

    isMatch = CompareHash(password, user.password)
    IF isMatch == FALSE:
        RETURN 'Invalid Credentials'

    token = GenerateJWT(user.id, user.role)
    RETURN token, user
```

16 Admin Login Flow

Job Creation Flow (Pseudocode):

This flow allows an authenticated admin to create a new job posting in the database. It ensures that only users with admin privileges can add jobs using role-based access control [5], [11].

```
//Job Creation Flow

FUNCTION CreateJob(token, jobData):
    admin = VerifyJWT(token)

    IF admin.role != 'admin':
        RETURN 'Permission Denied'

    newJob = InsertIntoDatabase(jobData)
    RETURN newJob
```

17 Job Creation Flow

Candidate Application Flow (Pseudocode):

This flow handles job applications submitted by candidates and records them securely in the database. It verifies the candidate's identity before storing application details [12], [5].

```
//Candidate Application Flow

FUNCTION ApplyForJob(token, jobId, resume):
    candidate = VerifyJWT(token)

    IF candidate.role != 'candidate':
        RETURN 'Unauthorized Access'

    application = SaveApplication(candidate.id, jobId, resume)
    RETURN application
```

18 Candidate Application Flow

Proctoring Event Flow (Pseudocode):

This flow captures webcam frames during the interview and analyzes them for facial landmarks, emotion, head pose, eye blinks, and mouth movements. It logs any detected violations for maintaining interview integrity.

```
//Proctoring Event Flow

FOR each captured webcam frame:
    landmarks = MediaPipeFaceMesh(frame)
    emotion = MiniXceptionPredict(frame)
    headPose = SolvePnP(landmarks)
    eyeBlink = ComputeEAR(landmarks)
    mouthMovement = ComputeMAR(landmarks)

    violations = CheckViolations(frame, landmarks, emotion)
    StoreViolations(violations)
```

19 Proctoring Event Flow

5.2. Components, Libraries, Web Services and stubs

The system integrates multiple components across the backend, frontend, and AI services:

• **Backend:** The server-side logic is implemented in Node.js using Express.js for API layer development. Database operations are handled via Mongoose ODM. Authentication is secured through JSON Web Token, while password hashing and verification use bcrypt [5], [11].

• **Frontend:** React.js is used for building the user interface, while Tailwind CSS is used for responsive styling and UI consistency [13], [14].

• **AI/ML Services:** Computer vision-based proctoring is implemented using MediaPipe FaceMesh for landmark detection, Mini-Xception CNN for emotion recognition, and YOLO/COCO-SSD for object detection [1], [3], [4], [6], [18].

• **Optimization Models:** Mobile-optimized neural networks such as MobileNet are used for efficient inference in real-time environments [6].

These components work together to ensure scalable, real-time interview monitoring and evaluation across distributed systems [5].

5.3. Deployment Environment

The deployment environment ensures modularity and scalability:

• **Backend:** The Node.js server runs on managed cloud platforms such as Render and Railway for fail-safe API hosting.

• **Database:** The MongoDB Atlas cloud database cluster securely stores all recruitment and interview process collections with encrypted access.

• **Frontend:** The React application is deployed on global frontend hosting platforms including Vercel and Netlify for scalable UI delivery and fast CDN-based load distribution.

• **ML Microservices:** AI monitoring services built on FastAPI are containerized using Docker for isolated scaling and deployed on cloud VM/container runtimes. The services are exposed on

secure internal ports (8001–8003) to process candidate interview frames and behavior signals through encrypted HTTP communication.

This deployment enables independent horizontal scaling of AI services, secure API-to-ML service communication, geographical accessibility, and high availability for concurrent interview sessions without performance compromise.

5.4. Tools and Techniques

The development process employed the following tools and techniques:

• **Development Tools:** Visual Studio Code for source development, Thunder Client for REST API testing, MongoDB Compass for local database inspection, GitHub for repository hosting, and Docker Desktop for AI microservice container testing and pipeline validation.

5.4.1. Techniques:

• Modular MVC architecture for backend organization, separating controllers, routes, models, and middleware for maintainability.
• REST API construction with structured JSON request-response layers.
• JWT-based authentication enforcing strict role-based access control for Admin/Recruiter and candidates.
• WebRTC webcam streaming integration for interview capture and monitoring.
• Asynchronous processing for minimal API latency and high parallel execution.
• Containerized ML microservices supporting isolated inference execution and independent scaling for recruitment monitoring AI.

5.5. Best Practices / Coding Standards

The implementation follows professional coding and security standards:

• **Code Organization:** Backend folders are structured into controllers, models, routes, middleware, and services, ensuring clear segregation between core recruitment logic and AI-monitoring microservices.

• **JavaScript Standards:** Development uses ES6+ conventions, reusable modules, async/await syntax for non-blocking execution, descriptive method naming, and structured response models for interview data handling.

Security Measures:

Passwords are hashed using bcrypt before storage.

Access control is enforced using JWT tokens with role payloads.

CORS settings restrict unauthorized origins.

All sensitive credentials and secrets are managed securely using environment variables via dotenv (.env).

No direct database access is allowed from the client/frontend.

• **Maintainability:** The platform supports independent updates for backend APIs and AI services without requiring full system redeployment.

• **Documentation:** Inline comments, modular function explanations, request-response descriptions, microservice stub contracts, and structured code significantly enhance readability for future development and recruiter process auditing.

5.6. Version Control

Version control is managed using **Git** and **GitHub**:

• **Branching Strategy:** The repository follows industry-standard branch segmentation including main (stable), development (integration), and separate feature branches for modules such as job publishing, interview session control, violation monitoring, and report generation.

• **Commit Practices:** All commits follow structured and meaningful messages for auditability and requirement traceability.

• **Collaboration:** Pull requests, peer code reviews, module approvals, and controlled merging ensure reliability and safety in feature integration.

• **Backup and Tracking:** GitHub cloud repository hosting provides persistent version history, deviation tracking, and secure rollback capability for both core recruitment and AI-monitoring development changes.

Chapter 6

Testing and Evaluation

Chapter 6: Testing and Evaluation

This chapter presents the complete testing and evaluation process carried out for the AI Proctor Interview System (Smartly Hire) to ensure it works accurately, securely, and smoothly for both recruiters (Admin) and candidates. The goal of testing was to confirm that every feature, including authentication, job creation, AI question generation, interview attempt submission, proctoring, and result evaluation, works according to the system requirements. Different testing approaches were applied to validate functionality, handle invalid input safely, and ensure secure data flow between frontend, backend, database, and AI services. The chapter also discusses system performance under normal and heavy usage conditions to verify reliability. Finally, stress testing was conducted to identify system limits and ensure stable behavior during unexpected high-load scenarios.

6.1 Use Case Testing

Use case testing was performed to validate the system based on real-life user workflows. This method ensures that the complete process works from start to end, rather than testing only isolated functions. In Smartly Hire, the two main user roles are Admin (Recruiter) and Candidate, therefore all use cases were tested from both perspectives.

For the Admin, use cases such as register/login, job posting creation, AI-based question generation using Ollama, viewing created jobs, viewing candidate attempts, and checking AI-evaluated results were tested. The system was checked to ensure that once an Admin logs in, their dashboard shows only the jobs and interview attempts linked to their account. Similarly, role-based access control was verified to confirm that only admins can access protected routes such as /admin/dashboard.

For the Candidate, use cases such as opening interview link, entering details, webcam-based proctoring activation, answering questions, and submitting the interview were tested. During evaluation, the system correctly recorded candidate answers, stored the attempt in MongoDB,

and generated results for the Admin dashboard. All use cases were validated using valid and invalid scenarios to confirm correct navigation flow and proper error handling.

6.2 Equivalence partitioning

Equivalence partitioning was used to reduce the number of test cases while ensuring strong coverage. In this method, input values are divided into logical groups (partitions), and one value from each partition is tested since all values behave similarly within the same group. This approach helped test form validation, API inputs, and database constraints efficiently.

For example, in the registration module, the email field was divided into valid and invalid partitions. Valid partition included properly formatted emails like user@gmail.com, while invalid partitions included missing @, missing domain, empty input, and random characters. Similarly, password input was tested using partitions such as valid password length, short password length, empty password, and weak passwords.

Equivalence partitioning was also applied in job creation forms (valid job title vs empty job title), interview attempt submission (valid answers vs empty answers), and proctoring events (valid webcam permission vs denied permission). This ensured the system handled all categories of input correctly without unnecessary repeated testing.

6.3 Boundary value analysis

Boundary value analysis was applied to identify errors that typically occur at the extreme limits of input values. Many software failures happen around boundary conditions such as minimum length, maximum length, and just outside allowed limits. This testing technique ensured that Smartly Hire accepts valid boundary values and rejects invalid ones.

For instance, the password field was tested at the minimum acceptable length and just below it. If the system required a minimum length of 6 characters, the test included inputs of 5 characters (invalid), 6 characters (valid), and 7 characters (valid). Similarly, text fields such as name and job title were tested using very short values, maximum allowed values, and excessive values to ensure the system does not crash or store incorrect data.

Boundary testing was also important in interview question generation. The number of questions requested was tested at small boundaries (e.g., 1 question), normal values (5 questions), and high values (10+ questions) to ensure the AI module responds properly and does not produce errors due to excessive load or long response times.

6.4 Data flow testing

Data flow testing was used to verify that the system maintains correct data movement from one component to another. Since Smartly Hire is a MERN-based system, data flows between React frontend, Express backend APIs, MongoDB database, and Ollama AI service. This testing ensured that data is created, stored, retrieved, and displayed correctly at every stage.

For example, when an Admin registers, the data flows from the registration form → backend register API → MongoDB users collection → success response back to frontend. During login, the JWT token is generated and returned to the frontend where it is stored in localStorage, then used for future protected requests. Another important data flow process includes job creation: admin submits job details → job stored in MongoDB with createdBy field linked to admin ID → admin dashboard fetches jobs and displays only relevant records.

In candidate interviews, data flow testing confirmed that the interview link loads job details, questions are displayed correctly, candidate answers are stored in MongoDB under interview attempts, and results are accessible on the admin side. This ensured that no data mismatches, missing values, or incorrect linking occurred in the system.

6.5 Unit testing

Unit testing was performed to verify the correctness of individual modules and functions independently. The purpose was to ensure each small unit works properly before integrating it with the whole system. Unit tests were mainly applied to backend functions such as controllers and middleware logic, as well as frontend components for correct rendering and behavior.

In the backend, unit testing covered features such as user registration validation, login authentication logic, password hashing, token generation, and role verification middleware. For example, the login controller was tested for scenarios like invalid email, incorrect password, and successful login. The middleware was also tested to confirm that requests without a token are rejected, and requests with invalid tokens are blocked.

On the frontend side, unit tests ensured that forms accept correct input, display error messages for invalid input, and show loading state when API calls are running. This improved overall system quality by catching small logical issues early, reducing future debugging effort.

6.6 Integration testing

Integration testing ensured that all modules work together as one complete system. This was essential because Smartly Hire involves multiple connected components: authentication, job management, AI question generation, interview attempt processing, and admin dashboards. The primary objective of integration testing was to validate that frontend requests reach the backend correctly, backend processes data and queries MongoDB correctly, and final responses are displayed properly.

For example, integration tests were performed on the Admin workflow: register → login → create job → generate AI questions → candidate attempts → admin views results. Another important integration scenario involved AI: backend sends request to Ollama API → receives generated questions → stores them or returns them → frontend renders them in interview page. These tests confirmed correct API endpoints, correct response formats, and secure role-based access.

JWT security integration was also verified, ensuring that protected routes require authentication and allow access only to valid user roles. If any request was made without token or with incorrect role, the system properly returned an error response.

6.7 Performance testing

Performance testing was conducted to measure how well the system performs under expected usage conditions. The platform includes time-sensitive operations such as login requests,

dashboard fetching, AI question generation, and interview submission. Performance evaluation focused on response speed, system stability, and user experience under normal traffic.

The system was tested to ensure that commonly used operations such as login, fetching jobs, and submitting interview attempts respond within acceptable time limits. AI question generation was also tested to ensure that responses from Ollama remain stable and do not delay the interview process significantly. Database performance was measured by repeatedly storing and fetching interview attempts to confirm that MongoDB queries remain efficient.

The results confirmed that the system runs smoothly for normal usage where few admins and candidates are active simultaneously. Performance testing ensured the system is responsive and stable in real-world recruitment usage.

6.8 Stress Testing

Stress testing was performed to evaluate system behavior when pushed beyond expected usage limits. The goal was to identify the maximum load the platform can handle and ensure the system fails safely without crashing or corrupting data. Stress testing is important for platforms like Smartly Hire because interviews can be conducted simultaneously by many candidates.

To test extreme conditions, multiple interview attempts were simulated together, with repeated candidate submissions, continuous dashboard refresh requests, and multiple AI question generation calls. The system was monitored to ensure that it continued working without breaking functionality. Under heavy load, response time naturally increased, but the system maintained correctness of stored records and continued accepting valid requests.

Stress testing proved that Smartly Hire can handle high user activity and heavy request volume with stable behavior. The system design ensures safe handling of overload situations by properly returning errors rather than crashing the entire application.

Chapter 7

Summary, Conclusion and Future Enhancements

Chapter 7: Summary, Conclusion & Future Enhancements

7.1. Project Summary

Smartly Hire was developed to address integrity challenges in remote recruitment, including impersonation and unauthorized assistance. The system is an AI-powered online interview platform that automates hiring while ensuring real-time proctoring. It uses a microservices architecture with React frontend, Node.js backend, MongoDB database, and Python AI services. Key features include role-based authentication, job management, and real-time candidate monitoring via face and emotion detection. The platform automatically generates integrity reports for recruiters and provides a fair interview experience for candidates. Overall, Smartly Hire delivers a scalable, secure, and automated solution for modern remote hiring needs.

7.2. Achievements and Improvements

The project successfully integrated multi-modal AI proctoring using MediaPipe FaceMesh, Mini-Xception CNN, and COCO-SSD models. A fully functional role-based dashboard was delivered for recruiters to manage jobs and view violations. Candidates benefit from a streamlined application process and real-time proctoring alerts during interviews. The platform demonstrated stable response times during performance testing with concurrent interview sessions. Key improvements included optimizing AI inference speed to 3-5 seconds per frame and implementing auto-submission mechanisms. The use of client-side ML models reduced backend processing load and improved overall system scalability.

7.3. Critical Review

Smartly Hire successfully integrates multiple AI proctoring techniques, addressing a significant gap in existing recruitment solutions. The microservices architecture provides good modularity and allows independent scaling of AI components across the platform. However, client-side AI processing can be inconsistent depending on the candidate's device hardware capabilities. Emotion detection accuracy may not capture nuanced or culturally specific expressions without further model fine-tuning. The system currently lacks live video streaming for recruiters to

monitor interviews in real-time. Overall, the system is robust but its effectiveness depends on consistent hardware and further model refinement.

7.4. Lessons Learnt

The team learned the importance of balancing AI model complexity with real-time performance for browser-based applications. Initially, heavier models caused latency issues, leading to the adoption of efficient mobile-optimized models. Implementing real-time proctoring required careful management of WebRTC resources and frame sampling intervals. The team discovered that well-documented REST APIs between backend and AI services were critical for debugging. Integration tasks required more time than estimated due to library compatibility issues. Clear version control practices proved essential for collaborative development across different system components.

7.5. Future Enhancements/Recommendations

To transform Smartly Hire into a truly inclusive and intelligent recruitment ecosystem, several key enhancements are proposed. Audio voice-to-text will be added, allowing interviewees to respond verbally while automatic transcription captures their answers. For color-blind users, the platform will introduce multiple accessible themes with high-contrast modes and customizable color palettes. The emotion detection model should be fine-tuned on diverse datasets to improve accuracy and reduce cultural bias. Integrating live streaming would allow recruiters to monitor interviews and intervene when severe violations occur. Server-side AI processing via GPU instances and a dedicated mobile application will further ensure consistent performance and accessibility across all devices.

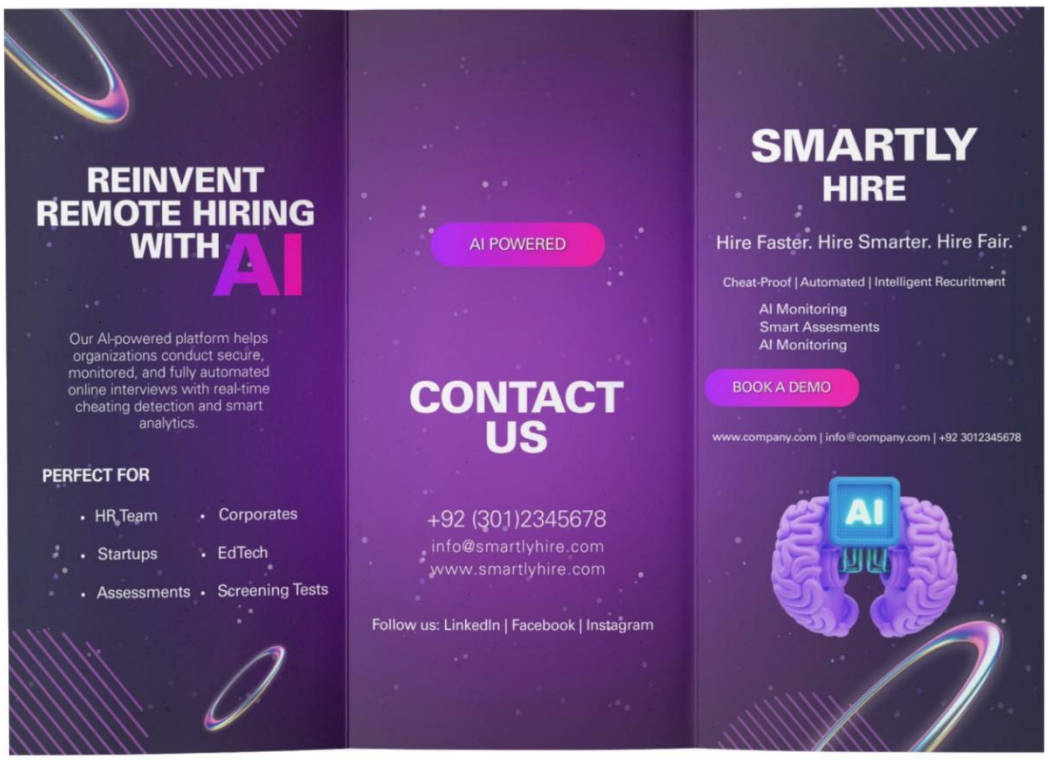
Appendices

Appendix A: Information / Promotional Material

This appendix presents the promotional and informational material developed for the **AI Proctor Interview System**. It demonstrates the branding, marketing, and awareness content prepared to communicate the system's features, capabilities, and benefits. The material includes brochures, flyers, standees, and banners, designed for both digital and physical distribution. These resources support user onboarding, recruitment campaigns, and organizational presentations, ensuring clear communication of the system's functionality to administrators, recruiters, and candidates. This appendix also includes guidelines for consistent use of colors, logos, and branding elements across all marketing and user-facing materials.

A.1. Broacher

The brochure provides a detailed overview of the AI Proctor Interview System. It highlights the system's core features, underlying technology stack, and key benefits for both administrators and candidates. The brochure is intended for distribution to organizations, educational institutions, and recruitment agencies. It visually summarizes the workflow of proctored interviews, AI-powered monitoring processes, and automated reporting capabilities, making it easy for stakeholders to understand the system at a glance.



20 Brochure Front side

21 Brochure inside

A.2. Flyer

The flyer is a one-page promotional document designed to attract both candidates and recruiters. It visually communicates the system's primary advantages, including secure interviews, real-time AI proctoring, and automated performance reporting. The flyer uses concise messaging and illustrative graphics to convey key features quickly. It is optimized for both digital distribution (emails, social media) and physical printing for events or notice boards.

22 Flyer

A.3. Standee

The standee is designed for display at events such as recruitment fairs, workshops, or conferences. It provides a visual summary of the system's capabilities, including candidate dashboards, AI monitoring interfaces, and real-time violation alerts. The standee uses diagrams

and infographics to demonstrate the interview workflow, capturing attention and conveying professionalism to stakeholders.

23 Standee

A.4. Branding Guidelines

This section outlines the visual identity and branding principles for the AI Proctor Interview System. It ensures consistency across all promotional material and user interfaces, enhancing brand recognition and professional presentation.

A.4.1. Color Scheme

The platform uses a sleek, modern color palette designed to communicate trust, sophistication, and advanced AI-driven technology.

Primary Colors

- **Deep Navy**: Establishes reliability, professionalism, and a stable visual foundation.
- **Purple–Blue Gradient**: Conveys innovation, intelligence, and a futuristic feel throughout the interface.
- **Soft White**: Ensures high readability and maintains a clean, minimalist aesthetic.

Accent Colors

Bright **pink-purple neon accents** are applied strategically to highlight key actions, such as primary buttons, navigation states, and important interactive elements. These accents provide contrast, guide user focus, and enhance the overall modern UI experience.

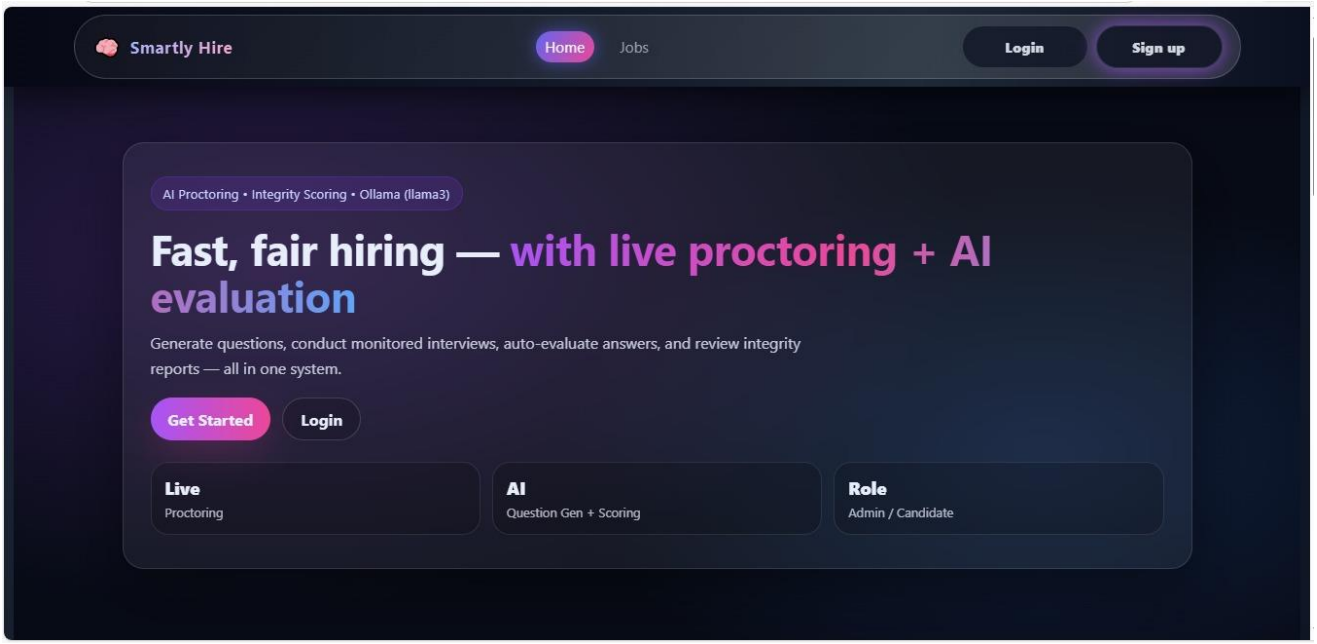
A.4.1.1. Logo Usage

The system logo is designed for versatility across digital and print media. Key guidelines include:

- Maintain clear spacing around the logo to avoid visual clutter
- Use consistent colors as defined in the color palette
- Preserve proportional scaling to avoid distortion
- Use the full-color version for digital applications and the monochrome version for printing purposes
- Do not alter or rotate the logo in any promotional or official materials

Appendix B: Sample Screenshots

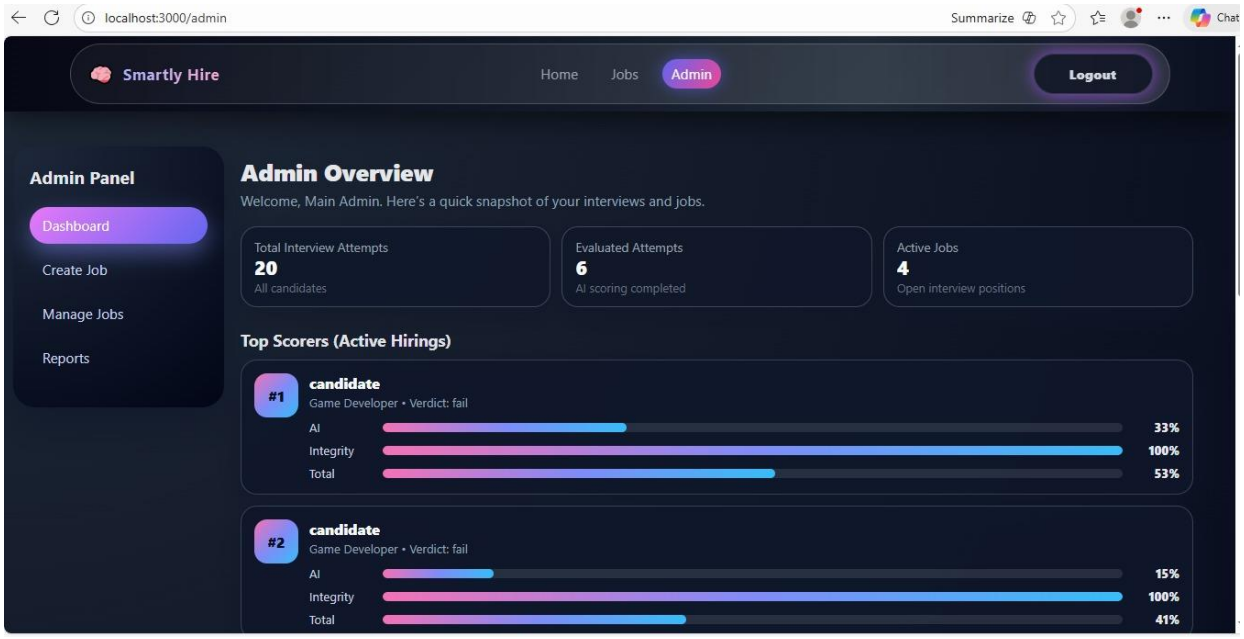
This appendix contains representative screenshots of the **AI Proctor Interview System** in operation. It demonstrates the user interface, administrative dashboards, candidate workflows, and AI-proctoring outputs. The appendix provides stakeholders with a clear understanding of the system's functionality, layout, and reporting capabilities. It also serves as evidence of the implemented features and system usability during interviews.



24 Landing page

B.1. Admin Dashboard Screenshots

The admin dashboard displays an overview of all job postings, candidate applications, and active interview sessions. It includes visual elements such as charts, cards, and tables that summarize key statistics, including total jobs, candidate counts, ongoing interviews, and violation metrics. These screenshots demonstrate the real-time monitoring and management capabilities available to administrators.



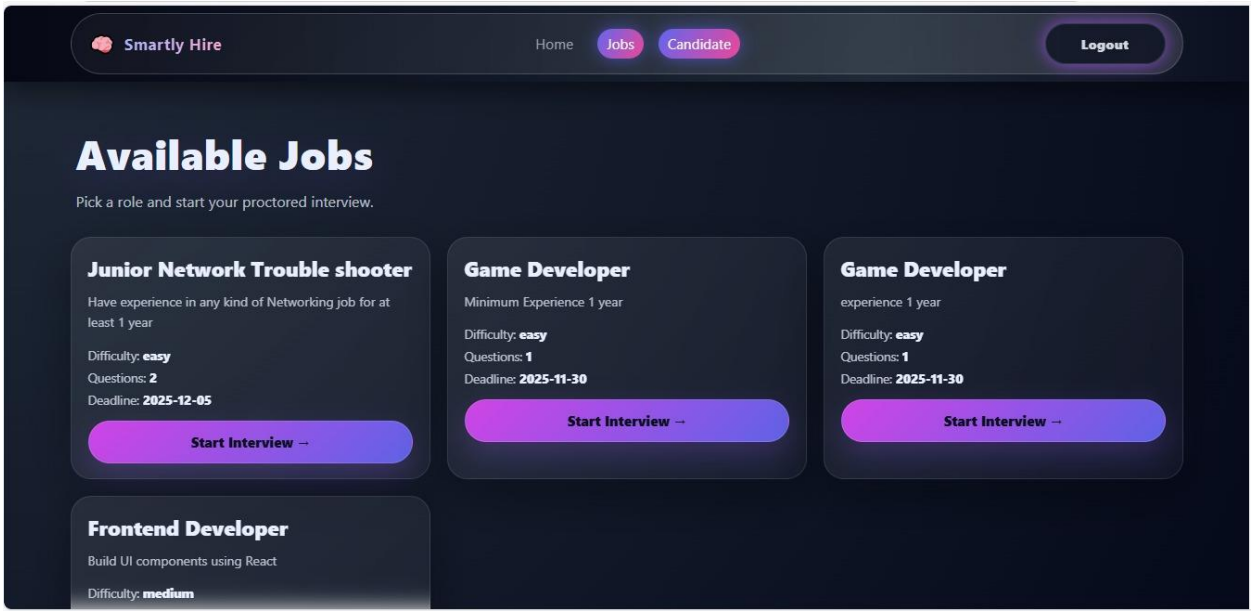
25 Admin Dashboard

B.1.1. Job Management Screens

This section shows screenshots of the job management interface, where administrators can create, edit, and delete job postings. Each job includes fields such as title, description, number of questions, and application deadlines. The interface provides filtering and search options for efficient job tracking and management.

B.1.1.1. Job Creation Form

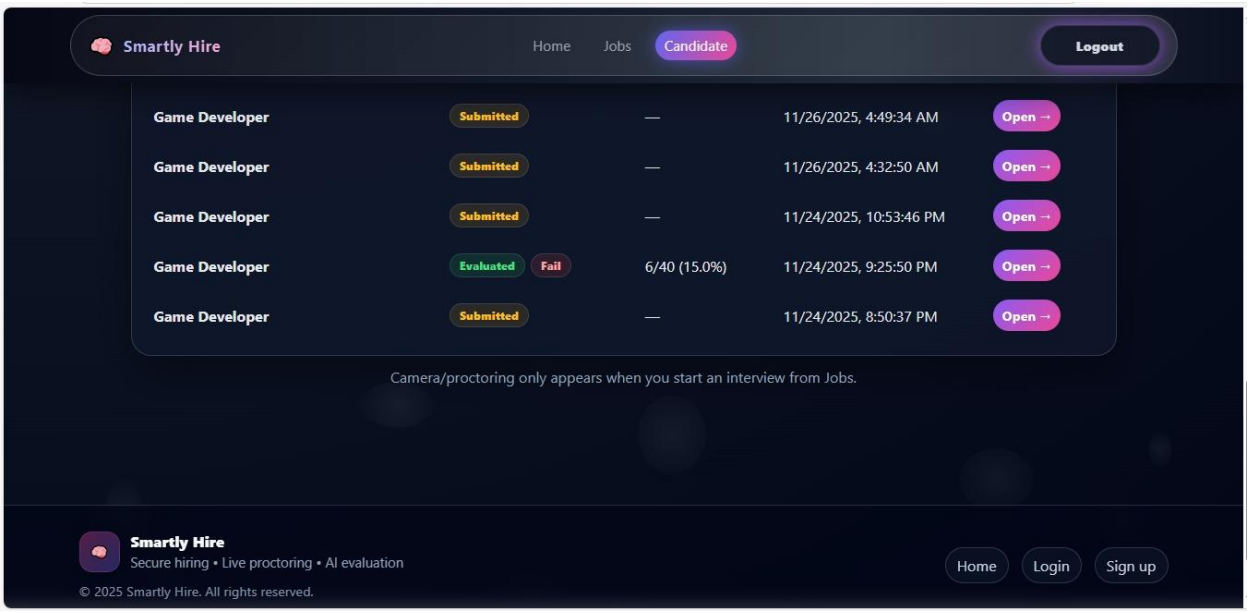
The job creation form allows administrators to input job details and generate AI-assisted job descriptions. It ensures that all necessary information is captured before posting the job, maintaining consistency and reducing manual effort.



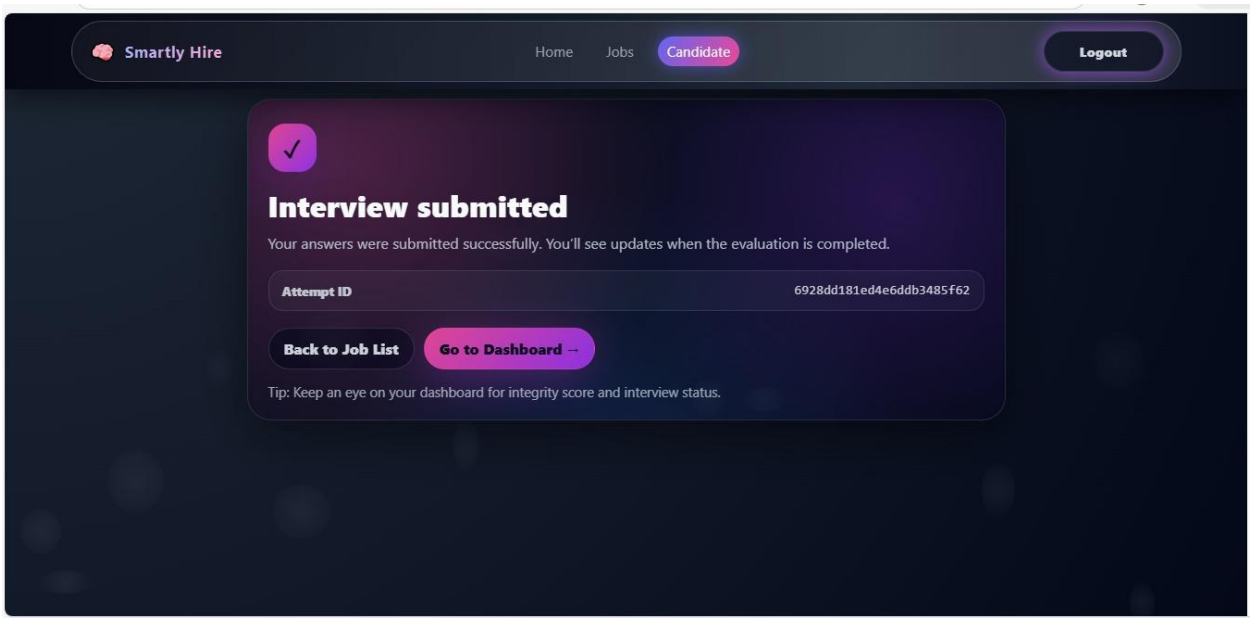
26 Available Jobs

B.2. Candidate Interface Screenshots

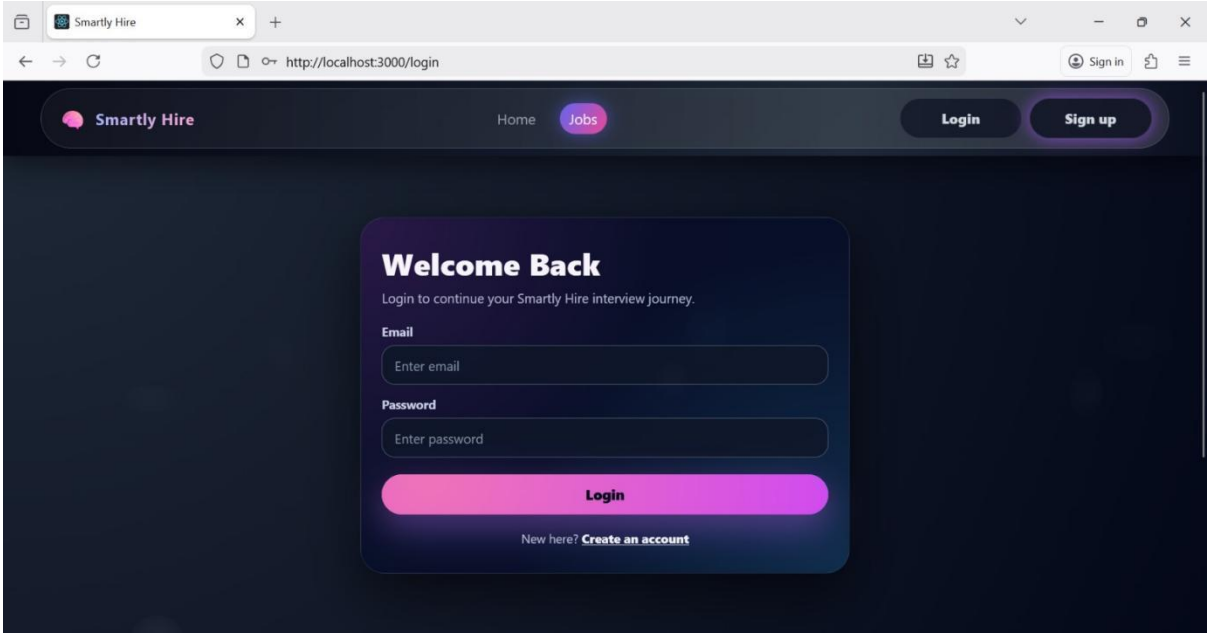
The candidate dashboard displays available job listings, applied positions, and scheduled interviews. It also shows the interview interface with questions, timers, and real-time proctoring alerts. These screenshots illustrate the candidate experience and interaction with the system.



27Candidate Interviews list



28 Interview Submitted Page



29 login page

Reference and Bibliography

References and Bibliography

[1] F. Chollet, "Xception: Deep Learning with Depthwise Separable Convolutions," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1251–1258.

[2] I. Bazarevsky et al., "BlazeFace: Sub-millisecond Neural Face Detection on Mobile GPUs," arXiv:1907.05047, 2019.

[3] Google, "MediaPipe: Cross-platform, customizable ML solutions," Available: https://developers.google.com/mediapipe

[4] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv:1804.02767, 2018.

[5] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2016.

[6] A. Howard et al., "MobileNets: Efficient CNNs for Mobile Vision Applications," arXiv:1704.04861, 2017.

[7] S. Li and W. Deng, "Deep Facial Expression Recognition: A Survey," *IEEE Transactions on Affective Computing*, vol. 13, no. 3, pp. 1195–1215, 2022.

[8] T. Baltrusaitis et al., "OpenFace: An Open Source Facial Behavior Analysis Toolkit," in *IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2016.

[9] P. Viola and M. Jones, "Rapid Object Detection using a Boosted Cascade," in *Proc. IEEE CVPR*, 2001.

[10] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proc. ICLR*, 2015.

[11] FastAPI, "FastAPI Documentation," https://fastapi.tiangolo.com/

[12] MongoDB Inc., "MongoDB Atlas Documentation," https://www.mongodb.com/cloud/atlas

[13] Facebook, "React Documentation," https://reactjs.org/

[14] Tailwind Labs, "Tailwind CSS Documentation," https://tailwindcss.com/

[15] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," in *Proc. IEEE CVPR*, 2005.

[16] A. Krizhevsky et al., "ImageNet Classification with Deep CNNs," *Communications of the ACM*, 2017.

[17] Z. Zhang, “Head Pose Estimation in Computer Vision: A Survey,” *IEEE TPAMI*, 2019.

[18] R. Girshick, “Fast R-CNN,” in *Proc. IEEE ICCV*, 2015.

[19] A. Rosebrock, “Deep Learning for Face Recognition,” PyImageSearch, 2020.

[20] J. Deng et al., “ImageNet: A Large-Scale Hierarchical Image Database,” in *Proc. IEEE CVPR*, 2009

Index

Index

[A]

Achievements, 7.2

Activity Diagram, 4.7

Admin Dashboard, 2.4.2, B.1

AI Proctoring, 2.4.6, 5.1

Architecture Diagram, 4.1

Audio Voice-to-Text, 7.5

[B]

Background, 1.1

Boundary Value Analysis, 6.3

Brochure, A.1

[C]

Class Diagram, 4.4

Color-blind Themes, 7.5

Component Diagram, 4.9

Critical Review, 7.3

[D]

Data Dictionary, 4.3

Data Flow Diagram, 4.11

Deployment Diagram, 4.10

Domain Model, 4.2

[E]

Empathy Map, 1.9

Entity Relationship Diagram, 4.3

Equivalence Partitioning, 6.2

Executive Summary, vii

[F]

Flyer, A.2

Functional Requirements, 2.4

Future Enhancements, 7.5

[G]

Gantt Chart, 1.7.3

Gap Analysis, 1.5

[I]

Implementation, 5

Integration Testing, 6.6

[L]

Lessons Learnt, 7.4

Literature Review, 1.4

[M]

MediaPipe, 1.6, 5.2

Microservices, 4.1, 5.2

Mobile Application, 7.5

[N]

Nonfunctional Requirements, 2.5

[O]

Operation Contracts, 4.6

[P]

Performance Testing, 6.7

Project Plan, 1.7

Proposed Solution, 1.6

[R]

References, 103

Report Generation, 2.4.8

Report Outline, 1.8

Roles & Responsibility Matrix, 1.7.2

[S]

Security Requirements, 2.5.3

Sequence Diagram, 4.5

Standee, A.3

State Transition Diagram, 4.8

Stress Testing, 6.8

[T]

Table of Contents, viii

Testing, 6

Tools & Techniques, 5.4

[U]

Unit Testing, 6.5

Use Case Analysis, 3

Use Case Testing, 6.1

User Authentication, 2.4.1

[V]

Version Control, 5.6

[W]

Work Breakdown Structure, 1.7.1